QUANTUM COMPUTING

COMPLETE EXAM PREPARATION GUIDE

University Exam Study Material

CHAPTERS COVERED

1. Foundations of Quantum Computing | 2. Quantum Systems & States
3. Quantum Gates & Circuits | 4. Quantum Algorithms
5. Quantum Communication | 6. Quantum Fourier Transform
7. Eigenvalues & Eigenvectors in Quantum Computing

CHAPTER 1: FOUNDATIONS OF QUANTUM COMPUTING

Quantum Computing is a paradigm of computation that harnesses the principles of quantum mechanics — superposition, entanglement, and interference — to process information in ways fundamentally different from classical computers. Classical computers use bits that are always either 0 or 1. Quantum computers use quantum bits (qubits) that can exist in a superposition of both 0 and 1 simultaneously.

1.1 Classical vs Quantum Computing

PROPERTY	CLASSICAL COMPUTING	QUANTUM COMPUTING
Bit (0 or 1, never both)	Qubit (superposition of 0 and 1)	
2 states for 1 bit, 2^n for n bits	Continuous superposition; 2^n amplitudes simultaneously	
Logic gates (AND, OR, NOT) — irreversible	Quantum gates — ALL reversible (unitary)	
One computation path at a time	Quantum parallelism: all paths simultaneously	
Not defined / not possible	Fundamental: amplitudes cancel or reinforce	
Bits can be copied freely	No-Cloning Theorem: quantum states cannot be copied	
Reads definite value without disturbance	Collapses superposition — destroys quantum state	
Sequential, reliable, everyday tasks	Exponential speedup for specific problems	
NOT gate (irreversible)	Hadamard gate H (reversible, creates superposition)	

1.2 The Qubit — Quantum Bit

A qubit is the smallest unit of quantum information. Unlike a classical bit that is strictly 0 or 1, a qubit exists in a superposition of both states simultaneously until it is measured.

QUBIT MATHEMATICAL DEFINITION
A qubit |psi> = c0|0> + c1|1> where c0, c1 are COMPLEX numbers (amplitudes) Basis states: |0> = [1, 0]^T   and |1> = [0, 1]^T
Normalization condition (MUST hold for quantum state): |c0|^2 + |c1|^2 = 1
Probability of measuring |0> = |c0|^2
Probability of measuring |1> = |c1|^2

The coefficients c0, c1 are called PROBABILITY AMPLITUDES

Unlike classical bits, a qubit can be ANYWHERE on the Bloch sphere surface

1.2.1 Normalization of Qubits

If a quantum state is given as |psi> = [2i+3, i-1]^T, we must first check if |2i+3|^2 + |i-1|^2 = 1. Since |2i+3|^2 = 4+9=13 and |i-1|^2 = 1+1=2, sum = 15 ≠ 1. So we normalize:

SOLVED NUMERICAL — Qubit Normalization

PROBLEM: Given |psi> = [2i+3, i-1]^T — is this a valid quantum state? If not, normalize it.

GIVEN: |psi> = [2i+3, i-1]^T

FIND: Check normalization; compute normalized state |psi'>

FORMULA: ||psi>||^2 = |c0|^2 + |c1|^2 must equal 1. Normalized: |psi'> = |psi> / ||psi>||

STEP 1: Compute |c0|^2 = |2i+3|^2 = (2i+3)(−2i+3) = 4+9 = 13

STEP 2: Compute |c1|^2 = |i-1|^2 = (i-1)(−i-1) = 1+1 = 2

STEP 3: Sum = 13 + 2 = 15 ≠ 1 → NOT a valid quantum state

STEP 4: ||psi>|| = sqrt(15)

STEP 5: Normalized state: |psi'> = (1/sqrt(15)) [2i+3, i-1]^T

STEP 6: Verify: |2i+3|^2/15 + |i-1|^2/15 = 13/15 + 2/15 = 15/15 = 1 ✓

FINAL ANSWER: |psi'> = [(2i+3)/sqrt(15), (i-1)/sqrt(15)]^T

VERIFY: |(2i+3)/sqrt(15)|^2 + |(i-1)/sqrt(15)|^2 = 13/15 + 2/15 = 1 ✓

1.3 Superposition

Superposition is the quantum principle that allows a system to exist in multiple states simultaneously. A qubit in superposition is simultaneously 0 and 1, with associated probability amplitudes. Only when measured does the system 'collapse' to a definite state.

Physical analogy: Schrodinger's cat — before you open the box, the cat is both alive AND dead. The act of observation (measurement) forces it into one state.

- Superposition is NOT the same as randomness or uncertainty — the system genuinely exists in multiple states
- The amplitudes carry phase information, enabling quantum interference
- Creating superposition: apply the Hadamard gate H to |0>: H|0> = (|0> + |1>)/sqrt(2)

1.4 State Representation — Vectors and Matrices

In quantum computing, the state of a system is represented as a column vector (ket vector in Dirac notation). The dynamics (how a state changes over time) is represented by a matrix. This is because dynamics changes one state (vector) to another — a transformation that requires a matrix, not another vector.

1.4.1 Deterministic Classical System

In a deterministic classical system, each state has exactly ONE outgoing edge (one-to-one mapping). The matrix representation M has exactly one 1 per column (permutation matrix). When M^2 is computed, each column still has at most one 1 — representing deterministic two-step dynamics.

DETERMINISTIC SYSTEM EXAMPLE
6 marbles arranged at positions 0,1,2,3,4,5 → state vector x = [6,2,1,5,3,10]^T
This is a column vector representing the arrangement of marbles
Matrix M represents dynamics: M[i,j] = 1 if there is an edge from state j to state i Property of M: each column has exactly ONE 1 (deterministic — one outgoing edge per state)
M*x gives the new state after one time step
M*M = M^2 → two time steps; still has at most one 1 per column

1.4.2 Probabilistic Classical System

In a probabilistic system, a marble can be at any of several positions with certain probabilities. The state vector now contains probabilities (non-negative reals summing to 1). The matrix M is now a STOCHASTIC matrix (column sums = 1). A doubly stochastic matrix has BOTH row sums and column sums equal to 1.

SOLVED NUMERICAL — Doubly Stochastic Matrix Verification
PROBLEM: Given M = [[0, 1/6, 5/6],[1/3, 1/2, 1/6],[2/3, 1/3, 0]] and state x = [1/6, 2/3, 1/6]^T, compute Mx and verify column sums.
GIVEN: M = [[0, 1/6, 5/6],[1/3, 1/2, 1/6],[2/3, 1/3, 0]], x = [1/6, 2/3, 1/6]^T
FIND: Mx = new state vector
FORMULA: Matrix multiplication: each entry of result = dot product of row of M with column vector x
STEP 1: Row 1 of Mx: 0*(1/6) + (1/6)*(2/3) + (5/6)*(1/6) = 0 + 2/18 + 5/36 = 4/36 + 5/36 = 9/36 = 1/4
STEP 2: Row 2 of Mx: (1/3)*(1/6) + (1/2)*(2/3) + (1/6)*(1/6) = 1/18 + 1/3 + 1/36 = 2/36 + 12/36 + 1/36 = 15/36 = 5/12
STEP 3: Row 3 of Mx: (2/3)*(1/6) + (1/3)*(2/3) + 0*(1/6) = 2/18 + 2/9 = 1/9 + 2/9 = 3/9 = 1/3
STEP 4: Sum of result: 1/4 + 5/12 + 1/3 = 3/12 + 5/12 + 4/12 = 12/12 = 1 ✓ (valid probability distribution)
STEP 5: Column sum check: col 1: 0+1/3+2/3 = 1; col 2: 1/6+1/2+1/3 = 1; col 3: 5/6+1/6+0 = 1 ✓
FINAL ANSWER: Mx = [1/4, 5/12, 1/3]^T — valid probability state. M is doubly stochastic.

VERIFY: Row sums and column sums of M^2 are also all 1 ✓

1.5 Quantum System — Complex Amplitudes & Interference

Quantum computing is based on quantum mechanics, and quantum mechanics is based on complex numbers. In quantum systems, the 'probabilities' are replaced by complex amplitudes c, and the actual probability is $|c|^2$. This is fundamentally different from classical probability.

QUANTUM vs CLASSICAL PROBABILITY
Classical: P(A) = 1/2, P(B) = 1/3 → P(A or B) = P(A) + P(B) = 1/2 + 1/3 = 5/6
Quantum: amplitude(A) = c_A, amplitude(B) = c_B
Combined amplitude = $c_A + c_B$ (amplitudes ADD, not probabilities)
Probability = $\|c_A + c_B\|^2$ which can be LESS than $\|c_A\|^2$ or $\|c_B\|^2$
Example: c1 = 5+3i → $\|c1\|^2$ = 25+9 = 34
c2 = -3-2i → $\|c2\|^2$ = 9+4 = 13
c1+c2 = 2+i → $\|c1+c2\|^2$ = 4+1 = 5 < 34 or 13 ← INTERFERENCE!
This is DESTRUCTIVE INTERFERENCE — amplitudes cancelling each other

1.5.1 Interference

Interference occurs when two quantum systems collide or combine. Their complex amplitudes add together. This can result in:

- CONSTRUCTIVE INTERFERENCE: amplitudes add up, probability increases
- DESTRUCTIVE INTERFERENCE: amplitudes cancel, probability decreases or becomes zero

In classical systems, interference cannot be defined because probabilities always add. Quantum algorithms exploit interference to amplify correct answers and cancel wrong answers.

1.6 Probabilistic Double Slit Experiment

The double slit experiment perfectly demonstrates the difference between classical probabilistic and quantum behavior. A bullet (classical) fired through two slits has probability 1/2 of going through each slit. The final distribution is simply the sum of individual probabilities — no interference pattern.

For quantum particles (electrons/photons): the particle goes through BOTH slits simultaneously (superposition). The probability amplitude from each slit interferes at the target. Where amplitudes add → bright fringe (constructive). Where they cancel → dark fringe (destructive). This interference pattern CANNOT be explained classically.

DOUBLE SLIT MATRIX EXAMPLE (from notes)
B = transition matrix for bullet going from gun → slits → target
Initial state of bullet: x = $[1,0,0,0,0,0,0,0]^T$ (bullet at position 0 = gun)

B*x = state after one click (bullet at slits)
B^2*x = B*B*x = state after two clicks (bullet at targets)
For quantum: amplitudes interfere → B^2 shows interference pattern
Classical: no interference, P(target_i) = sum of products along paths
Quantum: P(target_i) = |sum of amplitude products along paths|^2

1.7 Unitary Matrices — Quantum Operators

All quantum operations (gates) must be represented by UNITARY matrices. A unitary matrix U satisfies: U * U† = I = U† * U where U† (U-dagger) is the conjugate transpose of U.

COMPUTING U-DAGGER (Conjugate Transpose)
Step 1: Transpose the matrix (swap rows and columns)
Step 2: Take the complex conjugate of every entry (change sign of imaginary part: a+bi → a-bi) Example: If U = [[1/sqrt(2), 1/sqrt(2)], [-i/sqrt(2), i/sqrt(2)], [0, 0, i]]
Then U† = transpose of U with all i replaced by -i Verification: U*U† = I means U is unitary
Physical meaning: Unitary matrices PRESERVE the norm (probability) of quantum states A unitary matrix is automatically reversible — U^(-1) = U†

SOLVED NUMERICAL — Unitary Matrix Verification
PROBLEM: Verify that the quantum operator matrix M = [[1/sqrt(2), 1/sqrt(2), 0], [-i/sqrt(2), i/sqrt(2), 0], [0, 0, i]] is unitary.
GIVEN: M as given above
FIND: Check M*M† = I
FORMULA: Compute M† (conjugate transpose) then multiply M*M†
STEP 1: M† = [[1/sqrt(2), i/sqrt(2), 0], [1/sqrt(2), -i/sqrt(2), 0], [0, 0, -i]]
STEP 2: (M*M†)[0,0] = (1/sqrt2)(1/sqrt2) + (1/sqrt2)(1/sqrt2) + 0 = 1/2 + 1/2 = 1 ✓
STEP 3: (M*M†)[0,1] = (1/sqrt2)(i/sqrt2) + (1/sqrt2)(-i/sqrt2) + 0 = i/2 - i/2 = 0 ✓
STEP 4: (M*M†)[1,1] = (-i/sqrt2)(i/sqrt2) + (i/sqrt2)(-i/sqrt2) + 0 = 1/2 + 1/2 = 1 ✓
STEP 5: Off-diagonal elements: all = 0 ✓

STEP 6: Bottom right: $i*(-i) = -i^2 = 1$ ✓
FINAL ANSWER: M*M† = I → M is a unitary matrix ✓
VERIFY: Also verify M†*M = I for complete confirmation

TOPIC	CHAPTER 1 — FOUNDATIONS
DEFINITION	Quantum computing uses qubits (superposition states) and unitary operators; quantum mechanics is based on complex amplitudes.
KEY FORMULA / RULE	|psi> = c0|0>+c1|1>; |c0|^2+|c1|^2=1; P(outcome)=|amplitude|^2
TYPES / VARIANTS	Qubit, Superposition, Interference (constructive/destructive), Unitary matrix
KEY DIAGRAM	Bloch sphere for single qubit; Double slit experiment diagram
MOST LIKELY EXAM Q	Explain the difference between classical probabilistic and quantum systems using the double slit experiment. OR Verify a given matrix is unitary.
REMEMBER	Quantum interference arises from COMPLEX amplitudes — this is impossible in classical probability theory.

CHAPTER 2: QUANTUM SYSTEMS & STATES

This chapter covers how quantum systems are represented mathematically, how qubits are defined, how states are measured, and the concept of multiple qubits (multi-qubit systems). Understanding this chapter is fundamental to understanding quantum gates and algorithms.

2.1 Quantum System and Qubit — Formal Definition

A quantum system is any physical system that obeys the laws of quantum mechanics. The state of a quantum system is represented as a VECTOR in a complex vector space (Hilbert space). The qubit is the smallest quantum system — a 2-dimensional quantum system.

Property	Classical Bit	Qubit
Mathematical object	0 or 1 (scalar)	Vector in C^2
Basis states	|0> = [1,0]^T only or |1> = [0,1]^T only	|0> = [1,0]^T and |1> = [0,1]^T simultaneously
General state	Always |0> or |1> — no superposition	|psi> = c1|0> + c2|1>, c1,c2 in C
Normalization	N/A — always 0 or 1	|c1|^2 + |c2|^2 = 1 MUST hold
Measurement	Reads exact value, no disturbance	Collapses to |0> or |1>, destroys superposition
Information	1 bit per measurement	Can encode infinite information, but only 1 bit extractable per measurement
Physical realization	Transistor on/off	Electron spin, photon polarization, superconducting circuit

2.2 Dirac Notation (Bra-Ket Notation)

Dirac notation, also called bra-ket notation, is the standard mathematical language for quantum mechanics. It was invented by physicist Paul Dirac to elegantly express quantum states and operations.

DIRAC NOTATION COMPLETE REFERENCE
|psi> = KET = column vector = quantum state vector (carries the quantum state)
<psi| = BRA = |psi>† = row vector = complex conjugate transpose of ket <phi|psi> = INNER PRODUCT (bra-ket) = scalar = dot product in complex vector space
|psi><phi| = OUTER PRODUCT = matrix (n x n) ||psi>|| = NORM = sqrt(<psi|psi>) = length of vector
If the ball is at position 0: |x0> = [1,0,0,...,0]^T (standard basis vector)
If the ball is at position 1: |x1> = [0,1,0,...,0]^T

States |x0>, |x1>, ..., |x_{n-1}> are called BASIS STATES or CANONICAL STATES
Any state |psi> = c0|x0> + c1|x1> + ... = linear combination of basis states

2.2.1 Computing the Bra (<psi|) from the Ket (|psi>)

Given |psi> = [2i+2, 1-2i]^T:

- Transpose: [2i+2, 1-2i] (now a row vector)
- Complex conjugate of each entry: [(-2i+2), (1+2i)] (negate imaginary parts)
- So <psi| = [-2i+2, 1+2i]

2.3 Inner Product and Orthogonality

The inner product (dot product) of two quantum states gives a scalar that represents the 'overlap' between them. It is used to compute probabilities, check orthogonality, and compute norms.

SOLVED NUMERICAL — Inner Product Computation
PROBLEM: Compute <psi1|psi2> where |psi1> = [-2i+2, 1+2i] and |psi2> = [2i+2, 1-2i]^T
GIVEN: |psi1> = [2i+2, i-2]^T, so <psi1| = [-2i+2, 1+2i]
FIND: <psi1|psi2> (scalar inner product)
FORMULA: <psi1|psi2> = sum over k of (psi1_k)* * psi2_k
STEP 1: Convert |psi1> to <psi1|: transpose and conjugate → <psi1| = [-2i+2, 1+2i]
STEP 2: Element 1: (-2i+2)(2i+2) = -4i^2 - 4i + 4i + 4 = 4 + 4 = 8
STEP 3: Element 2: (1+2i)(1-2i) = 1 - 4i^2 = 1 + 4 = 5
STEP 4: <psi1|psi2> = 8 + 5 = 13
FINAL ANSWER: <psi1|psi2> = 13
VERIFY: Note: <psi|psi> = ||psi>||^2 = norm squared. This is also used to compute probability amplitudes.

ORTHOGONALITY: Two states |phi> and |psi> are orthogonal if <phi|psi> = 0. The basis states |0> and |1> are always orthogonal:

- <0|1> = [1,0][0,1]^T = 0 ✓ — basis states are orthogonal to each other
- <0|0> = [1,0][1,0]^T = 1 ✓ — inner product with self = 1 (normalized)

2.4 Measurement in Quantum Computing

Measurement is the process of extracting classical information from a quantum state. Before measurement, the qubit is in superposition. After measurement, the qubit COLLAPSES to one of the basis states. The probability of each outcome is determined by the Born rule.

BORN RULE — HOW MEASUREMENT WORKS
For state |psi> = c0|0> + c1|1>:
Probability of measuring |0> = |c0|^2 / (|c0|^2 + |c1|^2) [= |c0|^2 if normalized]
Probability of measuring |1> = |c1|^2 / (|c0|^2 + |c1|^2) [= |c1|^2 if normalized]
After measuring |0>: state collapses to |0>, the |1> component disappears
KEY: We CANNOT determine the quantum state from a single measurement
To estimate amplitudes: measure the SAME state preparation MANY TIMES (shots)
Measurement is IRREVERSIBLE — the superposition is permanently destroyed

SOLVED NUMERICAL — Measurement Probability Calculation
PROBLEM: Given |psi> = [-3-i, -2i, i, 2]^T (not normalized). Compute P(x2) and all basis state probabilities.
GIVEN: |psi> = [-3-i, -2i, i, 2]^T
FIND: P(x0), P(x1), P(x2), P(x3)
FORMULA: P(xi) = |ci|^2 / ||psi>||^2
STEP 1: |c0|^2 = |-3-i|^2 = 9+1 = 10
STEP 2: |c1|^2 = |-2i|^2 = 4
STEP 3: |c2|^2 = |i|^2 = 1
STEP 4: |c3|^2 = |2|^2 = 4
STEP 5: ||psi>||^2 = 10+4+1+4 = 19
STEP 6: P(x0) = 10/19, P(x1) = 4/19, P(x2) = 1/19, P(x3) = 4/19
STEP 7: Verification: 10/19 + 4/19 + 1/19 + 4/19 = 19/19 = 1 ✓
FINAL ANSWER: P(x2) = 1/19
VERIFY: Total probability = 1 ✓ (holds property of total probability)

2.5 Quantum Basis States and Canonical States

For a quantum system with n positions, the basis states are |x0>, |x1>, ..., |x_{n-1}>. These are standard unit vectors. Any arbitrary state |psi> is a linear combination (superposition) of these basis states. The basis states are also called CANONICAL STATES because they cannot be expressed as combinations of each other.

Example for 4-dimensional system: |x0> = [1,0,0,0]^T, |x1> = [0,1,0,0]^T, |x2> = [0,0,1,0]^T, |x3> = [0,0,0,1]^T

General state: |psi> = c0|x0> + c1|x1> + c2|x2> + c3|x3> = [c0,c1,c2,c3]^T

2.6 Equivalence of Quantum States

Two quantum states are EQUIVALENT (represent the same physical state) if one is a constant multiple of the other. Multiplying a state vector by a global phase factor (a complex number of magnitude 1) does NOT change any measurement probabilities.

CHECKING STATE EQUIVALENCE
Method: For |psi1> and |psi2>, compute alpha = (psi1_component_k / psi2_component_k) for each component k.
If all ratios are equal → states are equivalent (just differ by global phase or normalization).
Example: |psi1> = [2i+2, 1-2i]^T and |psi2> = [i+1, (1-2i)/2]^T
ratio_k1 = (2i+2)/(i+1) = 2(i+1)/(i+1) = 2 — same for both components
If ratios are all equal → same quantum state (|psi1> = 2*|psi2>)
Physical reason: |c*psi|^2 = |c|^2*|psi|^2 — if |c|=1, probabilities unchanged

2.7 Outer Product and Projectors

The outer product |psi><phi| is an N x M matrix formed by multiplying the ket |psi> (column) by the bra <phi| (row). This is used to define projection operators and to express quantum gates.

OUTER PRODUCT EXAMPLES
|0><0| = [1,0]^T * [1,0] = [[1,0],[0,0]] — projects onto |0> state
|0><1| = [1,0]^T * [0,1] = [[0,1],[0,0]]
|1><0| = [0,1]^T * [1,0] = [[0,0],[1,0]]
|1><1| = [0,1]^T * [0,1] = [[0,0],[0,1]] — projects onto |1> state Completeness relation: |0><0| + |1><1| = Identity matrix I
Outer product |psi><psi| is always a projector: (|psi><psi|)^2 = |psi><psi|

2.8 Tensor Product — Multi-Qubit Systems

To describe systems with multiple qubits, we use the TENSOR PRODUCT (symbol: ⊗). The tensor product of two state vectors creates a larger state vector describing the combined system. For two qubits of dimensions n1 and n2, the combined system has dimension n1 × n2.

TENSOR PRODUCT COMPUTATION
For vectors: [a,b]^T ⊗ [c,d]^T = [a*c, a*d, b*c, b*d]^T

<table>
<tr><td>For matrices: [a,b]^T ⊗ [x,y,z]^T = [a*x, a*y, a*z, b*x, b*y, b*z]^T</td></tr>
<tr><td>|00> = |0>⊗|0> = [1,0]^T ⊗ [1,0]^T = [1,0,0,0]^T</td></tr>
<tr><td>|01> = |0>⊗|1> = [1,0]^T ⊗ [0,1]^T = [0,1,0,0]^T</td></tr>
<tr><td>|10> = |1>⊗|0> = [0,1]^T ⊗ [1,0]^T = [0,0,1,0]^T</td></tr>
<tr><td>|11> = |1>⊗|1> = [0,1]^T ⊗ [0,1]^T = [0,0,0,1]^T</td></tr>
<tr><td>For n qubits: 2^n basis states needed → system grows EXPONENTIALLY</td></tr>
<tr><td>POSITION MATTERS in tensor product: |0>⊗|1> ≠ |1>⊗|0></td></tr>
</table>

2.8.1 Two-Qubit States and Separability

A two-qubit state |psi> = c0|00> + c1|01> + c2|10> + c3|11> is SEPARABLE if it can be written as a tensor product of two single-qubit states |psi1>⊗|psi2>. If it CANNOT be written as a tensor product, the state is ENTANGLED.

<table>
<tr><th>SOLVED NUMERICAL — Two-Qubit Tensor Product</th></tr>
<tr><td>PROBLEM: Given |psi1> = alpha_1|0> + beta_1|1> and |psi2> = alpha_2|0> + beta_2|1>, compute |psi1>⊗|psi2>.</td></tr>
<tr><td>GIVEN: |psi1> = alpha_1|0> + beta_1|1>, |psi2> = alpha_2|0> + beta_2|1></td></tr>
<tr><td>FIND: |psi1>⊗|psi2> expanded in basis {|00>,|01>,|10>,|11>}</td></tr>
<tr><td>FORMULA: Distribute tensor product over addition</td></tr>
<tr><td>STEP 1: |psi1>⊗|psi2> = (alpha_1|0> + beta_1|1>) ⊗ (alpha_2|0> + beta_2|1>)</td></tr>
<tr><td>STEP 2: = alpha_1*alpha_2 |0>⊗|0> + alpha_1*beta_2 |0>⊗|1> + beta_1*alpha_2 |1>⊗|0> + beta_1*beta_2 |1>⊗|1></td></tr>
<tr><td>STEP 3: = alpha_1*alpha_2 |00> + alpha_1*beta_2 |01> + beta_1*alpha_2 |10> + beta_1*beta_2 |11></td></tr>
<tr><td>STEP 4: So: c0 = alpha_1*alpha_2, c1 = alpha_1*beta_2, c2 = beta_1*alpha_2, c3 = beta_1*beta_2</td></tr>
<tr><td>STEP 5: Note: c0*c3 = alpha_1*alpha_2*beta_1*beta_2 = c1*c2 ← separability condition</td></tr>
<tr><td>FINAL ANSWER: |psi1>⊗|psi2> = alpha_1*alpha_2|00> + alpha_1*beta_2|01> + beta_1*alpha_2|10> + beta_1*beta_2|11></td></tr>
<tr><td>VERIFY: For separable state: c0*c3 = c1*c2 always holds</td></tr>
</table>

2.9 Entangled Qubits

Entanglement is one of the most counter-intuitive and powerful features of quantum mechanics. An entangled state of two qubits cannot be written as a tensor product of individual qubit states. Knowing the state of one qubit instantly determines the state of the other, regardless of their physical separation (Einstein called this 'spooky action at a distance').

PROOF THAT |psi> = |00> + |11> IS ENTANGLED

Suppose it were separable: |psi> = (c0|0>+c1|1>) ⊗ (c0'|0>+c1'|1>)

Expanding: c0*c0'|00> + c0*c1'|01> + c1*c0'|10> + c1*c1'|11>

Matching with |psi> = 1*|00> + 0*|01> + 0*|10> + 1*|11>:

c0*c0' = 1 (so c0 ≠ 0 and c0' ≠ 0)

c0*c1' = 0 → c1' = 0 (since c0 ≠ 0)

c1*c0' = 0 → c1 = 0 (since c0' ≠ 0)

c1*c1' = 1 → but c1=0 makes c1*c1'=0 ≠ 1 — CONTRADICTION!

Therefore |00> + |11> CANNOT be written as tensor product → it is ENTANGLED ✓

2.10 Measurement on Multi-Qubit Systems

When measuring a multi-qubit state, we may measure all qubits simultaneously or just one qubit at a time. Partial measurement (measuring only one qubit) collapses that qubit and leaves the remaining qubits in a (possibly different) normalized state.

SOLVED NUMERICAL — Partial Measurement of Multi-Qubit State

PROBLEM: Given |psi> = (1/sqrt(2))|00> + (1/sqrt(2))|10> + (1/sqrt(3))|11>. Find prob. of measuring first qubit as |0>. What is the state after measurement?

GIVEN: |psi> = (1/sqrt(2))|00> + (1/sqrt(2))|10> + (1/sqrt(3))|11>

FIND: P(first qubit = |0>); state after measurement

FORMULA: Sum |amplitude|^2 for all terms where first qubit is |0>

STEP 1: Terms with first qubit = |0>: only (1/sqrt(2))|00>

STEP 2: P(first qubit = |0>) = |1/sqrt(2)|^2 / (|1/sqrt(2)|^2 + |1/sqrt(2)|^2 + |1/sqrt(3)|^2)

STEP 3: Denominator = 1/2 + 1/2 + 1/3 = 4/3

STEP 4: P(first qubit = |0>) = (1/2) / (4/3) = (1/2)*(3/4) = 3/8

STEP 5: After measuring |0> on first qubit, remaining state = |0> ⊗ |0> = |00>

STEP 6: New state = |psi_new> = |0> ⊗ (1/sqrt(2))|0> / sqrt(3/8) ... normalized to |00>

FINAL ANSWER: P(first qubit = |0>) = 3/8. After measurement: |psi_new> = |0>⊗|0> = |00>

VERIFY: The remaining qubit collapses to |0> because only the |00> term had first qubit as |0>

TOPIC	CHAPTER 2 — QUANTUM SYSTEMS & STATES
DEFINITION	Quantum states are complex vectors; qubits are 2D quantum systems; measurement collapses superposition according to Born rule.
KEY FORMULA / RULE	P(xi) = |ci|^2 / ||psi>||^2; Normalization: sum|ci|^2 = 1; <phi|psi> = inner product
TYPES / VARIANTS	Ket |psi>, Bra <psi|, Inner product, Outer product, Tensor product, Entanglement
KEY DIAGRAM	Bloch sphere; Basis states |0>,|1>; Tensor product of two qubits
MOST LIKELY EXAM Q	Prove that |psi> = |00>+|11> cannot be written as a tensor product (show entanglement). OR Compute measurement probabilities for a given multi-qubit state.
REMEMBER	After measuring one qubit, RENORMALIZE the remaining state — the norm changes after partial measurement.

CHAPTER 3: QUANTUM GATES & CIRCUITS

Quantum gates are the fundamental building blocks of quantum circuits. Every quantum gate is a unitary matrix. Unlike classical gates (which can be irreversible, e.g., AND gate), ALL quantum gates are reversible. This is because quantum evolution must be unitary, and all unitary matrices have inverses.

3.1 Single-Qubit Gates — Pauli Gates

Pauli gates are the three fundamental single-qubit gates named after physicist Wolfgang Pauli. Together with the identity gate, they form the Pauli group. All quantum gates on single qubits can be expressed using combinations of Pauli gates.

3.1.1 X Gate (Pauli-X / NOT Gate / Inverter)

The X gate is the quantum equivalent of the classical NOT gate. It flips |0> to |1> and |1> to |0>. It is represented by the matrix:

<table>
<tr><th>X GATE (PAULI-X / QUANTUM NOT)</th></tr>
<tr><td>Matrix: X = [[0, 1], [1, 0]]</td></tr>
<tr><td>Action: X|0> = |1> and X|1> = |0> (bit flip)
Circuit symbol: ─[X]─ or ─[⊕]─</td></tr>
<tr><td>X|0> = [[0,1],[1,0]] [1,0]^T = [0,1]^T = |1> ✓</td></tr>
<tr><td>X|1> = [[0,1],[1,0]] [0,1]^T = [1,0]^T = |0> ✓</td></tr>
<tr><td>Property: X^2 = I (applying X twice returns to original state)</td></tr>
<tr><td>Eigenvalues: +1 and -1; Eigenvectors: |+> = (|0>+|1>)/sqrt(2) and |-> = (|0>-|1>)/sqrt(2)</td></tr>
</table>

3.1.2 Y Gate (Pauli-Y)

The Y gate performs a bit flip AND a phase flip simultaneously. It introduces a factor of i when flipping states.

<table>
<tr><th>Y GATE (PAULI-Y)</th></tr>
<tr><td>Matrix: Y = [[0, -i], [i, 0]]</td></tr>
<tr><td>Action: Y|0> = i|1> and Y|1> = -i|0></td></tr>
<tr><td>Y|0> = [[0,-i],[i,0]] [1,0]^T = [0,i]^T = i|1></td></tr>
<tr><td>Y|1> = [[0,-i],[i,0]] [0,1]^T = [-i,0]^T = -i|0></td></tr>
<tr><td>Effect: Flips bit (0→1, 1→0) AND changes phase (multiplies by ±i)
Property: Y^2 = I; Eigenvalues: +1 and -1</td></tr>
</table>

3.1.3 Z Gate (Pauli-Z / Phase Flip Gate)

The Z gate does NOT flip the bit — it leaves |0> unchanged but multiplies |1> by -1 (phase flip). It only acts non-trivially on the |1> component.

Z GATE (PAULI-Z)
Matrix: Z = [[1, 0], [0, -1]]
Action: Z|0> = |0> and Z|1> = -|1> (phase flip only, no bit flip)
Z|0> = [[1,0],[0,-1]] [1,0]^T = [1,0]^T = |0> ✓
Z|1> = [[1,0],[0,-1]] [0,1]^T = [0,-1]^T = -|1> ✓
Property: Z^2 = I; Eigenvalues: +1 (for |0>) and -1 (for |1>)
In Bloch sphere: rotates by pi around Z-axis

3.1.4 I Gate (Identity Gate / Buffer Gate)

The Identity gate does nothing — it maps every state to itself. It is included for completeness in mathematical descriptions.

I GATE (IDENTITY)
Matrix: I = [[1, 0], [0, 1]] Action: I|psi> = |psi> for any state |psi>
Used in tensor products: I⊗X means 'apply I to first qubit, X to second qubit'
H⊗I⊗I means 'apply H to first qubit, nothing to second and third'

3.2 Hadamard Gate — The Superposition Creator

The Hadamard gate is arguably the most important single-qubit gate. It creates SUPERPOSITION from a basis state, and is used in virtually every quantum algorithm to initialize qubits into uniform superposition.

HADAMARD GATE (H / SUPERPOSITION GATE)
Matrix: H = (1/sqrt(2)) [[1, 1], [1, -1]]
H|0> = (1/sqrt(2))[1,1]^T = (|0> + |1>)/sqrt(2) = |+> (uniform superposition)
H|1> = (1/sqrt(2))[1,-1]^T = (|0> - |1>)/sqrt(2) = |->
H applied twice returns to original: H*H = I (self-inverse)
H⊗n applied to |0...0> creates equal superposition of all 2^n basis states

H^⊗2 |00> = (|00>+|01>+|10>+|11>)/2 (uniform superposition of 4 states)
Used at beginning of Simon's algorithm, Grover's algorithm, QFT, Simon's problem
H maps Z-basis to X-basis: H|0>=|+>, H|1>=|->; and X-basis to Z-basis: H|+>=|0>, H|->=|1>

SOLVED NUMERICAL — Hadamard Gate Application
PROBLEM: Apply H to |1>. Write the result as a superposition and compute probabilities of measuring |0> and |1>.
GIVEN: |1> = [0, 1]^T
FIND: H|1> = ?
FORMULA: H|1> = (1/sqrt(2)) [[1,1],[1,-1]] [0,1]^T
STEP 1: H|1> = (1/sqrt(2)) [1*0 + 1*1, 1*0 + (-1)*1]^T
STEP 2: = (1/sqrt(2)) [1, -1]^T
STEP 3: = (1/sqrt(2))|0> + (-1/sqrt(2))|1>
STEP 4: = (|0> - |1>)/sqrt(2) = |->
STEP 5: P(|0>) = |1/sqrt(2)|^2 = 1/2
STEP 6: P(|1>) = |-1/sqrt(2)|^2 = 1/2
STEP 7: Total probability = 1/2 + 1/2 = 1 ✓
FINAL ANSWER: H|1> = (|0> - |1>)/sqrt(2) = |->. P(|0>) = P(|1>) = 1/2
VERIFY: H|0> = |+> = (|0>+|1>)/sqrt(2), H|1> = |-> = (|0>-|1>)/sqrt(2)

3.3 Phase Gates — S Gate and T Gate

Phase gates add a phase to the |1> component while leaving |0> unchanged. They are used in quantum algorithms like QFT and for fault-tolerant quantum computing.

Gate	Matrix	Action	Phase Added
S gate (Phase gate)	[[1,0],[0,i]]	S|0>=|0>, S|1>=i|1>	Phase pi/2 (90 degrees)
T gate (pi/8 gate)	[[1,0],[0,e^(i*pi/4)]]	T|0>=|0>, T|1>=e^(i*pi/4)|1>	Phase pi/4 (45 degrees)
S† (S-dagger)	[[1,0],[0,-i]]	S†|0>=|0>, S†|1>=-i|1>	Phase -pi/2
T† (T-dagger)	[[1,0],[0,e^(-i*pi/4)]]	T†|0>=|0>, T†|1>=e^(-i*pi/4)|1>	Phase -pi/4

3.4 Two-Qubit Gates

Two-qubit gates operate on two qubits simultaneously. They are represented by 4x4 unitary matrices (since two qubits have 4 basis states: |00>, |01>, |10>, |11>). The most important two-qubit gate is the CNOT gate.

3.4.1 CNOT Gate (Controlled NOT Gate)

The CNOT gate has two qubits: a CONTROL qubit (q1) and a TARGET qubit (q0). If the control qubit is |1>, the target qubit is flipped (X applied). If the control qubit is |0>, the target qubit is unchanged. This implements conditional operations — crucial for creating entanglement.

CNOT GATE COMPLETE REFERENCE
Matrix (4x4): CNOT = [[1,0,0,0],[0,1,0,0],[0,0,0,1],[0,0,1,0]]
Action on all basis states: CNOT|00> = |00> (control=0, target unchanged)
CNOT|01> = |01> (control=0, target unchanged)
CNOT|10> = |11> (control=1, target flipped: 0→1) CNOT|11> = |10> (control=1, target flipped: 1→0)
Circuit: q1 is control (with dot •), q0 is target (with ⊕ circle)
CNOT is its own inverse: CNOT^2 = I (applying twice returns to original) Creating entanglement: CNOT (H⊗I |00>) = CNOT (|+>|0>) = (|00>+|11>)/sqrt(2) = Bell state

SOLVED NUMERICAL — CNOT Gate on Superposition Input
PROBLEM: Apply CNOT to the state |psi> = (|0>+|1>)/sqrt(2) ⊗ |0>. Show the input, action, and output.
GIVEN: |psi_in> = (|0>+|1>)/sqrt(2) ⊗ |0> = (|00>+|10>)/sqrt(2)
FIND: CNOT|psi_in> = ?
FORMULA: CNOT flips target (q0) if control (q1) is |1>
STEP 1: CNOT|psi_in> = CNOT (|00>+|10>)/sqrt(2)
STEP 2: = (CNOT|00> + CNOT|10>)/sqrt(2)
STEP 3: = (|00> + |11>)/sqrt(2)   ← because CNOT|10>=|11> (control=1, target 0→1)
STEP 4: This is the Bell state |Phi+> = (|00>+|11>)/sqrt(2)
STEP 5: This state is ENTANGLED — cannot be written as tensor product of two qubits
STEP 6: P(|00>) = 1/2, P(|11>) = 1/2

FINAL ANSWER: CNOT((|0>+|1>)/sqrt(2) ⊗ |0>) = (|00>+|11>)/sqrt(2) = Bell state |Phi+>
VERIFY: This is the standard circuit for creating Bell states: apply H to first qubit, then CNOT

3.4.2 SWAP Gate

The SWAP gate exchanges the states of two qubits. SWAP|01> = |10> and SWAP|10> = |01>. It leaves |00> and |11> unchanged.

SWAP GATE
Matrix: SWAP = [[1,0,0,0],[0,0,1,0],[0,1,0,0],[0,0,0,1]]
SWAP|00> = |00>, SWAP|01> = |10>, SWAP|10> = |01>, SWAP|11> = |11> SWAP can be implemented using 3 CNOT gates: SWAP = CNOT_12 * CNOT_21 * CNOT_12
Used in QFT to reverse the bit order of the output

3.5 Three-Qubit Gates

Three-qubit gates operate on three qubits and are represented by 8x8 unitary matrices (2^3 = 8 basis states). The two most important are the Toffoli gate and the Fredkin gate.

3.5.1 Toffoli Gate (CCNOT — Controlled Controlled NOT)

The Toffoli gate has TWO control qubits and ONE target qubit. The target is flipped ONLY if BOTH control qubits are |1>. It is sometimes called the CCX gate. The Toffoli gate is UNIVERSAL — any classical Boolean function can be implemented using Toffoli gates. It is also reversible and quantum-mechanically valid.

TOFFOLI GATE
Mapping: |x,y,z> → |x,y,z⊕(x∧y)>
x=0,y=0: target unchanged |0,0,z> → |0,0,z>
x=1,y=0: target unchanged |1,0,z> → |1,0,z>
x=0,y=1: target unchanged |0,1,z> → |0,1,z>
x=1,y=1: target FLIPPED |1,1,z> → |1,1,z⊕1> = |1,1,NOT(z)>
Matrix: 8x8 identity with bottom-right 2x2 block being [[0,1],[1,0]] (X gate)
UNIVERSALITY: AND = Toffoli with z=0 (output in target), NOT = X gate
Toffoli is REVERSIBLE and UNIVERSAL for classical and quantum computation Quantum universal gates: {S,T,H,CNOT} or {T,H,CNOT}

3.5.2 Fredkin Gate (Controlled SWAP Gate)

The Fredkin gate (also called CSWAP — Controlled SWAP) has ONE control qubit and TWO target qubits. When the control is |1>, the two target qubits are swapped. When the control is |0>, nothing changes. Like the Toffoli gate, the Fredkin gate is also UNIVERSAL.

FREDKIN GATE (CONTROLLED SWAP)
Mapping: |x,y,z> → |x,z,y> if x=1; or |x,y,z> if x=0
000 → 000, 001 → 001, 010 → 010, 011 → 011
100 → 100, 101 → 110, 110 → 101, 111 → 111
Circuit symbol: control qubit (•) with two SWAP symbols (×) on target wires
Like Toffoli, Fredkin is reversible and universal

3.6 Quantum Universal Gates

A set of quantum gates is UNIVERSAL if any quantum computation (unitary operation) can be approximated to arbitrary accuracy using gates from that set. Just as classical computation has universal gates (NAND or NOR), quantum computation has universal gate sets.

Universal Gate Set	Gates Included	Why Universal
{S, T, H, CNOT}	Phase gate S, pi/8 gate T, Hadamard H, CNOT	Together approximate any unitary matrix to arbitrary precision
{T, H, CNOT}	pi/8 gate T, Hadamard H, CNOT	Slightly simpler set; also achieves universality
{Toffoli}	Toffoli (CCNOT) gate only	Universal for classical (reversible) computation; ALMOST universal quantum
{Fredkin}	Fredkin (CSWAP) gate only	Universal for classical computation; also conserves number of 1s

3.7 No-Cloning Theorem

The No-Cloning Theorem is a fundamental theorem of quantum mechanics stating that it is IMPOSSIBLE to create an identical copy of an arbitrary unknown quantum state. This has profound implications for quantum communication and cryptography.

NO-CLONING THEOREM — PROOF SKETCH
Goal: Does there exist a unitary C (cloning operator) such that C(|psi>⊗|blank>) = |psi>⊗|psi>?
Suppose C is linear (as all quantum operations must be): C(f(x)+f(y)) = f(C(x))+f(C(y))
C(|psi>⊗|0>) = |psi>⊗|psi> — suppose this works for specific states |x> and |y>

C(|x>⊗|0>) = |x>⊗|x> and C(|y>⊗|0>) = |y>⊗|y>
By linearity: C((|x>+|y>)/sqrt(2) ⊗ |0>) = (|x>⊗|x> + |y>⊗|y>)/sqrt(2)
But we WANT: (|x>+|y>)/sqrt(2) ⊗ (|x>+|y>)/sqrt(2) = (|xx>+|xy>+|yx>+|yy>)/2
These two expressions are NOT equal → Cloning as a linear (quantum) gate CANNOT EXIST! Consequence: You must DESTROY your qubit to send it (used in teleportation protocol)

3.8 Bell States — Maximally Entangled Two-Qubit States

Bell states are four specific two-qubit quantum states that are maximally entangled. They form a complete orthonormal basis for the two-qubit Hilbert space. Named after physicist John Stewart Bell. They are the foundation of quantum teleportation and superdense coding.

Bell State	Formula	Circuit to Create	Properties
|Phi+>	(|00> + |11>)/sqrt(2)	H on qubit 1, then CNOT(1→0)	Symmetric, even parity
|Phi->	(|00> - |11>)/sqrt(2)	H on qubit 1, Z on result, then CNOT	Z phase on |11> term
|Psi+>	(|01> + |10>)/sqrt(2)	H on qubit 1, CNOT, X on qubit 0	Antisymmetric, odd parity
|Psi->	(|01> - |10>)/sqrt(2)	H on qubit 1, CNOT, X and Z on qubit 0	Maximally entangled, odd parity

3.8.1 Bell State Circuit (from professor notes)

The standard circuit to create Bell state |Phi+>: Start with |00>, apply H to first qubit giving (|0>+|1>)/sqrt(2) ⊗ |0> = (|00>+|10>)/sqrt(2), then apply CNOT: (|00>+|11>)/sqrt(2) = |Phi+>. This circuit creates entanglement from unentangled inputs.

3.9 Comparison of All Single-Qubit Gates

Gate	Matrix	X Action	Y Action	Eigenvalues	Use Case
X (NOT)	[[0,1],[1,0]]	0→1, 1→0	Bit flip	+1, -1	Bit flip, initialize |1>
Y	[[0,-i],[i,0]]	i*1, -i*0	Bit+phase flip	+1, -1	Rotation in Bloch sphere
Z	[[1,0],[0,-1]]	0→0, 1→-1	Phase flip	+1, -1	Phase kick, Z-measurement
H	1/sqrt2 [[1,1],[1,-1]]	(0+1)/s2	(0-1)/s2	+1, -1	Create superposition

S	[[1,0],[0,i]]	0→0, 1→i*1	Add pi/2 phase	+1, i	Phase gate, QFT
T	[[1,0],[0,e^ipi/4]]	0→0, 1→e^ipi/4*1	Add pi/4 phase	+1, e^ipi/4	Universal gate, QFT
I	[[1,0],[0,1]]	0→0, 1→1	No change	+1, +1	Placeholder, completeness

TOPIC	CHAPTER 3 — QUANTUM GATES & CIRCUITS
DEFINITION	Quantum gates are reversible unitary matrices; classical gates are irreversible. Key gates: X, Y, Z, H, S, T, CNOT, Toffoli, Fredkin.
KEY FORMULA / RULE	H = (1/sqrt2)[[1,1],[1,-1]]; CNOT = flip target iff control=|1>; X=[[0,1],[1,0]]
TYPES / VARIANTS	Single-qubit: X,Y,Z,H,S,T,I; Two-qubit: CNOT, SWAP; Three-qubit: Toffoli (CCNOT), Fredkin (CSWAP)
KEY DIAGRAM	CNOT circuit diagram; Bell state creation circuit (H then CNOT); Toffoli gate truth table
MOST LIKELY EXAM Q	Explain the No-Cloning Theorem and prove it using linearity. OR Describe all Pauli gates and their matrices with worked examples. OR Create Bell states using quantum circuits.
REMEMBER	ALL quantum gates are reversible (unitary). Classical gates like AND are NOT reversible. Only NOT gate is reversible classically.

CHAPTER 4: QUANTUM ALGORITHMS

Quantum algorithms exploit superposition, entanglement, and interference to solve certain computational problems exponentially faster than classical algorithms. This chapter covers the three major quantum algorithms taught in your course: Grover's Algorithm, Simon's Algorithm, and quantum algorithm fundamentals.

4.1 Quantum Oracle (Black-Box Function)

A quantum oracle is a quantum gate that implements a classical function f in a reversible way. The oracle takes an input register and an ancilla (helper) register and XOR's the output into the ancilla: $U_f|x\rangle|y\rangle = |x\rangle|y \oplus f(x)\rangle$

QUANTUM ORACLE Uf
$U_f|x\rangle|y\rangle = |x\rangle|y \oplus f(x)\rangle$ where $\oplus$ is XOR (addition modulo 2)
Special case: if y = $|0\rangle$, then $U_f|x\rangle|0\rangle = |x\rangle|f(x)\rangle$ (writes f(x) into ancilla)
The oracle Uf is UNITARY (reversible) because XOR is its own inverse Applying oracle twice returns to original: $U_f(U_f|x\rangle|y\rangle) = U_f|x\rangle|y\oplus f(x)\oplus f(x)\rangle = |x\rangle|y\rangle$ In quantum algorithms: apply oracle to SUPERPOSITION of all inputs simultaneously
This achieves quantum parallelism: evaluating f(x) for all x at once

4.2 Grover's Search Algorithm

Grover's algorithm solves the unstructured search problem: given a database of N items, find the one specific item x0 marked by an oracle. Classical search requires O(N) queries on average. Grover's algorithm solves this in O(sqrt(N)) queries — a quadratic speedup.

4.2.1 Problem Setup

- We have $N = 2^n$ items labeled 0 to N-1
- Oracle f(x) = 1 if x = x0 (target), else f(x) = 0
- Classical search: O(N/2) queries on average, O(N) worst case
- Grover's quantum search: $O(\sqrt{N}) = O(\sqrt{2^n}) = O(2^{n/2})$ queries
- For N=4 (2 qubits): classical needs up to 4 queries, Grover needs just 1 iteration!
- For $N=10^6$: classical needs 500,000 queries, Grover needs ~1000 queries

4.2.2 Grover's Algorithm — Step-by-Step

GROVER'S ALGORITHM COMPLETE STEPS
STEP 0: Initialize register to $|0^n\rangle|1\rangle$ (n data qubits + 1 ancilla qubit)
STEP 1: Apply $H^{\otimes n}$ to data qubits and H to ancilla → uniform superposition:

|psi_1> = (1/sqrt(2^n)) * sum_x|x> * (|0>-|1>)/sqrt(2)
STEP 2: Apply oracle Uf → phase kickback marks the target state x0 with -1 phase:
|psi_2> = (1/sqrt(2^n)) * sum_x (-1)^f(x) |x> * (|0>-|1>)/sqrt(2)
Effect: target state x0 gets amplitude flipped from +1/sqrt(N) to -1/sqrt(N)
STEP 3: Apply Inversion about the Mean (also called Grover diffusion operator -I+2A):
v' = -v + 2*mean(v) = -I+2A applied to amplitude vector This amplifies the target and suppresses non-targets
STEP 4: Repeat Steps 2-3 for sqrt(N) iterations
STEP 5: Measure — probability of finding x0 approaches 1

4.2.3 Phase Inversion (Oracle Step)

The oracle marks the target by flipping its amplitude from positive to negative. All other amplitudes remain positive. The ancilla qubit trick: if we start with ancilla = (|0>-|1>)/sqrt(2), then after oracle:

- For x ≠ x0: |x>(|0>-|1>)/sqrt(2) → +|x>(|0>-|1>)/sqrt(2) (unchanged)
- For x = x0: |x0>(|0>-|1>)/sqrt(2) → -|x0>(|0>-|1>)/sqrt(2) (phase flipped)

This is called PHASE KICKBACK — the phase from the ancilla kicks back to the control qubit. After phase inversion, all amplitudes are still 1/sqrt(N) but the target has a NEGATIVE sign.

4.2.4 Inversion About the Mean

After phase inversion, one amplitude is negative. The inversion-about-the-mean operation reflects all amplitudes around their average. Since the target is negative and below the mean, after reflection it goes high above the mean. Non-target amplitudes are slightly above mean, so after reflection they go slightly below mean.

SOLVED NUMERICAL — Inversion About the Mean — Numerical Example
PROBLEM: Given 5 amplitudes v = [10, 10, 10, -10, 10] (target is index 3 which is marked). Apply inversion about the mean.
GIVEN: v = [10, 10, 10, -10, 10]^T
FIND: v' after inversion about mean
FORMULA: v'_i = 2*mean(v) - v_i = a + (a - v_i) where a = mean(v)
STEP 1: Compute mean: a = (10+10+10-10+10)/5 = 30/5 = 6
STEP 2: Apply v'_i = 2*a - v_i for each element:
STEP 3: v'[0] = 2*6 - 10 = 12 - 10 = 2 (suppressed)
STEP 4: v'[1] = 2*6 - 10 = 2 (suppressed)

STEP 5: v'[2] = 2*6 - 10 = 2 (suppressed)
STEP 6: v'[3] = 2*6 - (-10) = 12 + 10 = 22 (AMPLIFIED — this is the target!)
STEP 7: v'[4] = 2*6 - 10 = 2 (suppressed)
STEP 8: Result: v' = [2, 2, 2, 22, 2] — target amplitude increased from -10 to 22!
FINAL ANSWER: v' = [2, 2, 2, 22, 2]. Target (index 3) amplified to 22, others suppressed to 2.
VERIFY: Inversion about mean = (-I + 2A)v where A is the all-1/N matrix. A^2=A so (-I+2A)(-I+2A)=I ✓ (unitary)

4.2.5 Number of Iterations

Grover's algorithm requires exactly sqrt(N) = sqrt(2^n) iterations to maximize the probability of finding the target. Too few iterations: target probability not maximized. Too many iterations: probability starts to decrease (the algorithm 'overshoots').

GROVER ITERATION COUNT
Number of iterations = sqrt(N) = sqrt(2^n) = 2^(n/2) For n=1 (N=2): sqrt(2) ≈ 1 iteration
For n=2 (N=4): sqrt(4) = 2 iterations... but actually 1 gives ~1 success For n=5 (N=32): sqrt(32) = sqrt(25) = 5 iterations: iterate sqrt(25) = sqrt(32) ≈ 5 times
Matrix representation of Grover diffusion: -I + 2A where A[i,j] = 1/N for all i,j A is idempotent: A^2 = A
(-I+2A)(-I+2A) = I - 4A + 4A^2 = I - 4A + 4A = I → diffusion is unitary ✓

4.2.6 Grover Algorithm Circuit Diagram

The full Grover circuit for n=2 qubits:

GROVER'S ALGORITHM CIRCUIT
Input: |0>^⊗n (data register, n qubits) and |1> (ancilla qubit)
|0>—[H^⊗n]————[Uf]————[-I+2A]———[measure] |1>—[H]———————————|———————————————————————
  State evolution:

|psi_0> = |0^n>|1> (initialized)
|psi_1> = (H^⊗n ⊗ H)|psi_0> = (sum_x|x>/sqrt(N)) ⊗ (|0>-|1>)/sqrt(2)
|psi_2> = Uf|psi_1> = (sum_x (-1)^f(x)|x>/sqrt(N)) ⊗ (|0>-|1>)/sqrt(2)
|psi_3> = (-I+2A)|psi_2> = amplified target state
Repeat Uf and (-I+2A) for sqrt(N) total times
Measurement: target x0 found with high probability

4.3 Simon's Algorithm

Simon's problem: Given a 2-to-1 function f: {0,1}^n → {0,1}^n where f(x) = f(y) iff x = y⊕c for some hidden period c, find c. Classically requires O(2^(n/2)) queries. Simon's quantum algorithm uses only O(n) quantum queries plus O(n^2) classical post-processing — exponential speedup!

4.3.1 Simon's Problem — Key Concepts

- f is a 2-to-1 function: every output has exactly 2 inputs that map to it
- f(x) = f(y) if and only if x = y ⊕ c (x XOR c = y)
- The period c is an n-bit string: c ∈ {0,1}^n
- Goal: Find c using as few oracle queries as possible
- Example: if c = 101, then f(000)=f(101), f(001)=f(100), f(010)=f(111), etc.

4.3.2 Simon's Quantum Algorithm Steps

SIMON'S ALGORITHM — FULL PROCEDURE
STEP 0: |psi_0> = |0^n>|0^n> (two n-qubit registers, both initialized to 0) STEP 1: Apply H^⊗n to first register: |psi_1> = (1/sqrt(2^n)) sum_x |x>|0^n> STEP 2: Apply oracle Uf: |psi_2> = (1/sqrt(2^n)) sum_x |x>|f(x)>
STEP 3: Apply H^⊗n to first register: |psi_3> involves interference
|psi_3> = sum_z sum_x (-1)^<z,x> |z>|f(x)> / 2^n where <z,x> = z·x = bitwise dot product mod 2
STEP 4: Measure first register → get some z If <z,c> = 0: measurement gives z with probability 2/2^n
If <z,c> = 1: measurement gives z with probability 0 (destructive interference!)
STEP 5: Repeat O(n) times to get n-1 linearly independent equations z_i · c = 0 STEP 6: Solve the linear system classically to find c

SOLVED NUMERICAL — Simon's Algorithm — Example
PROBLEM: f: {0,1}^3 → {0,1}^3 with c = 101. Verify f(000)=f(101), f(001)=f(100), etc.
GIVEN: c = 101, n = 3, so 000 to 111 (8 inputs map to 4 unique outputs)
FIND: Verify 2-to-1 property and show groupings
FORMULA: For each x, compute x XOR c = x XOR 101. Those pairs share f value.
STEP 1: 000 XOR 101 = 101 → {000, 101} share same f value
STEP 2: 001 XOR 101 = 100 → {001, 100} share same f value
STEP 3: 010 XOR 101 = 111 → {010, 111} share same f value
STEP 4: 011 XOR 101 = 110 → {011, 110} share same f value
STEP 5: From professor's notes: f(000)=f(101)=010, f(001)=f(100)=000 (example values)
STEP 6: Quantum algorithm gives equations: z · c = 0 mod 2
STEP 7: If we measure z=011: 0*1+1*0+1*1 = 0+0+1 = 1 → so c has z·c=1 → z=011 would not appear
STEP 8: Valid z's (orthogonal to c=101): z satisfying z1+z3=0 mod 2
FINAL ANSWER: c = 101 found by solving the system of linear equations z·c = 0 mod 2
VERIFY: Need n-1 = 2 linearly independent z vectors to uniquely determine c for n=3

4.4 Complexity Comparison

Algorithm	Problem	Classical Complexity	Quantum Complexity	Speedup Type
Grover's Search	Unstructured database search among N items	O(N) queries	O(sqrt(N)) queries	Quadratic speedup
Simon's Algorithm	Find period c of 2-to-1 function	O(2^(n/2)) queries	O(n) quantum + O(n^3) classical	Exponential speedup
QFT (Shor's)	Integer factoring, period finding	O(exp(n^(1/3))) classical	O(n^2 log n)	Exponential speedup
Deutsch-Jozsa	Constant vs balanced function	O(2^(n-1)) queries	1 quantum query	Exponential speedup

TOPIC	CHAPTER 4 — QUANTUM ALGORITHMS
DEFINITION	Quantum algorithms use superposition to evaluate all inputs simultaneously, then interference to amplify the correct answer.

KEY FORMULA / RULE	Grover: O(sqrt(N)) queries; Simon: O(n) quantum queries; Diffusion operator: -I+2A
TYPES / VARIANTS	Grover's Search, Simon's Algorithm, Phase kickback, Inversion about mean, Quantum oracle Uf
KEY DIAGRAM	Grover circuit diagram; Simon circuit with two registers and oracle Uf
MOST LIKELY EXAM Q	Describe Grover's algorithm with circuit, state evolution at each step, and explain how inversion about the mean amplifies the target. OR Explain Simon's algorithm and its exponential speedup over classical.
REMEMBER	Grover's inversion-about-mean is (-I+2A). It is UNITARY because A^2=A → (-I+2A)(-I+2A)=I.

CHAPTER 5: QUANTUM COMMUNICATION

Quantum communication uses quantum mechanical phenomena to transmit information in ways impossible with classical communication. The two main protocols — Quantum Teleportation and Superdense Coding — are both based on EPR pairs (entangled Bell states). This chapter is extremely important for your exam.

5.1 EPR Pairs and Bell States (Recap)

EPR pairs (named after Einstein-Podolsky-Rosen) are pairs of maximally entangled qubits. The four Bell states serve as the four possible EPR pairs.

Bell State Name	Formula	Properties
|Phi+> (phi-plus)	(|00> + |11>)/sqrt(2)	Even parity, symmetric
|Phi-> (phi-minus)	(|00> - |11>)/sqrt(2)	Even parity, phase difference
|Psi+> (psi-plus)	(|01> + |10>)/sqrt(2)	Odd parity, symmetric
|Psi-> (psi-minus)	(|01> - |10>)/sqrt(2)	Odd parity, antisymmetric (singlet state)

5.2 Quantum Teleportation

Quantum teleportation transmits an arbitrary unknown quantum state |psi> = a|0> + b|1> from Alice to Bob using: (1) a shared EPR pair, (2) 2 classical bits of communication, and a classical channel. Important: the original qubit is DESTROYED in the process (no cloning!). The state is TELEPORTED, not the physical particle.

QUANTUM TELEPORTATION — KEY PROPERTIES
1. Communicates: The QUANTUM STATE (not the particle itself)
2. Classical bits sent: 2 (Alice measures her two qubits and sends results to Bob)
3. Channel used: Classical channel for the 2 measurement bits
4. Quantum channel: EPR pair shared in advance
5. No faster-than-light communication (Bob must wait for Alice's classical bits) 6. Original qubit is DESTROYED (No-Cloning Theorem compliance)
7. Bob must apply a correction gate based on Alice's 2 classical bits

5.2.1 Teleportation Protocol — Full Mathematical Derivation

Alice wants to send |psi> = a|0> + b|1> to Bob. They share EPR pair |Phi+> = (|00>+|11>)/sqrt(2). Alice has first qubit of EPR pair, Bob has second.

SOLVED NUMERICAL — Quantum Teleportation Full Derivation

PROBLEM: Alice teleports |psi> = a|0>+b|1> to Bob using shared |Phi+>. Show all steps.

GIVEN: |psi> = a|0>+b|1>, |Phi+> = (|00>+|11>)/sqrt(2). Alice has qubit 1 of EPR pair.

FIND: Final state in Bob's register equals a|0>+b|1> after Bob's correction

FORMULA: Tensor product + CNOT + Hadamard + measurement

STEP 1: Total 3-qubit state: |psi>⊗|Phi+> = (a|0>+b|1>) ⊗ (|00>+|11>)/sqrt(2)

STEP 2: = (1/sqrt2)(a|000>+a|011>+b|100>+b|111>)

STEP 3: Step 1 (Alice): Apply CNOT with Alice's qubit as control, her EPR qubit as target:

STEP 4: = (1/sqrt2)(a|000>+a|011>+b|110>+b|101>)

STEP 5: Step 2 (Alice): Apply H to Alice's first qubit:

STEP 6: H|0> = (|0>+|1>)/sqrt2, H|1> = (|0>-|1>)/sqrt2

STEP 7: = (1/2)(a(|0>+|1>)|00> + a(|0>+|1>)|11> + b(|0>-|1>)|10> + b(|0>-|1>)|01>)

STEP 8: = (1/2)(|00>(a|0>+b|1>) + |01>(a|1>+b|0>) + |10>(a|0>-b|1>) + |11>(a|1>-b|0>))

STEP 9: Step 3 (Alice): Measure her 2 qubits → one of 4 outcomes

STEP 10: If Alice measures |00>: Bob has a|0>+b|1> = |psi> (no correction needed → I)

STEP 11: If Alice measures |01>: Bob has a|1>+b|0> → apply X gate to get a|0>+b|1>

STEP 12: If Alice measures |10>: Bob has a|0>-b|1> → apply Z gate to get a|0>+b|1>

STEP 13: If Alice measures |11>: Bob has a|1>-b|0> → apply XZ gate to get a|0>+b|1>

FINAL ANSWER: Bob applies {I, X, Z, XZ} depending on Alice's 2 classical bits {00,01,10,11}

VERIFY: In all cases, Bob recovers exactly |psi> = a|0>+b|1> ✓

5.2.2 Teleportation Correction Table

Alice Measures	Bob's Uncorrected State	Bob Applies	Bob's Final State
|00>	a|0>+b|1>	I (identity, do nothing)	a|0>+b|1> = |psi> ✓
|01>	a|1>+b|0>	X gate (bit flip)	a|0>+b|1> = |psi> ✓
|10>	a|0>-b|1>	Z gate (phase flip)	a|0>+b|1> = |psi> ✓
|11>	a|1>-b|0>	XZ gate (both)	a|0>+b|1> = |psi> ✓

5.3 Superdense Coding

Superdense coding is the reverse of teleportation in a sense. It allows Alice to send 2 CLASSICAL bits of information to Bob by transmitting only 1 qubit, using a pre-shared EPR pair. This is the maximum classical information that can be encoded in a qubit under these conditions.

SUPERDENSE CODING KEY PROPERTIES

1. Information sent: 2 CLASSICAL bits (not quantum)
2. Channel: 1 QUANTUM channel (Alice sends her qubit to Bob)
3. Pre-shared resource: EPR pair |Phi+> = (|00>+|11>)/sqrt(2)
4. Alice and Bob share |Phi+>: Alice has first qubit, Bob has second
5. Alice encodes 2 classical bits by applying ONE of four gates to her qubit
6. Alice sends her qubit to Bob → Bob now has both qubits
7. Bob decodes by applying CNOT then H, then measuring both qubits

5.3.1 Superdense Coding Encoding Table

Alice applies one of four gates to encode 2 classical bits into her qubit:

<table>
<tr><th>Alice's 2 Bits</th><th>Alice Applies</th><th>State After Alice's Gate</th><th>Bob Receives Bell State</th></tr>
<tr><td>00</td><td>I (identity)</td><td>|Phi+> = (|00>+|11>)/sqrt(2)</td><td>|Phi+></td></tr>
<tr><td>01</td><td>X gate (bit flip)</td><td>X⊗I |Phi+> = (|10>+|01>)/sqrt(2) = |Psi+></td><td>|Psi+></td></tr>
<tr><td>10</td><td>Z gate (phase flip)</td><td>Z⊗I |Phi+> = (|00>-|11>)/sqrt(2) = |Phi-></td><td>|Phi-></td></tr>
<tr><td>11</td><td>XZ gate (both)</td><td>XZ⊗I |Phi+> = (|10>-|01>)/sqrt(2) = |Psi-></td><td>|Psi-></td></tr>
</table>

5.3.2 Bob's Decoding (Cases from Professor Notes)

Bob applies CNOT (his qubit as target, received qubit as control) then H to received qubit, then measures both qubits. Here are all four cases:

SOLVED NUMERICAL — Superdense Coding Decoding — All 4 Cases

PROBLEM: Bob receives one qubit from Alice; Bob already has second qubit. Show decoding for case: Alice sent '00'.

GIVEN: Bob has |Phi+> = (|00>+|11>)/sqrt(2) (Alice sent I, so state unchanged)

FIND: Decode to recover Alice's 2 classical bits '00'

FORMULA: Apply CNOT (Alice's qubit control, Bob's qubit target), then H to Alice's qubit

STEP 1: State: (|00>+|11>)/sqrt(2)

STEP 2: Apply CNOT (first qubit=control, second=target):

STEP 3: CNOT(|00>+|11>)/sqrt(2) = (|00>+|10>)/sqrt(2) = (|0>+|1>)/sqrt(2) ⊗ |0>

STEP 4: Apply H to first qubit:

STEP 5: H*(|0>+|1>)/sqrt(2) = H|+> = |0>

STEP 6: State after H: |0>|0> = |00>

STEP 7: Measure both qubits: get |00> → Alice's bits were '00' ✓

STEP 8: Case '01': |Psi+>→CNOT→(|01>+|11>)/sqrt(2)→H→|0>|1>=|01> → read '01' ✓

STEP 9: Case '10': |Phi->→CNOT→(|00>-|10>)/sqrt(2)→H→|1>|0>=|10> → read '10' ✓

STEP 10: Case '11': |Psi->→CNOT→(|01>-|11>)/sqrt(2)→H→|1>|1>=|11> → read '11' ✓

FINAL ANSWER: Bob's measurement result directly gives Alice's 2 classical bits in all cases.

VERIFY: The CNOT followed by H is the Bell measurement — it converts Bell states to computational basis states.

5.4 Quantum Basis States and Measurement in Different Bases

Quantum states can be expressed and measured in different bases. The three standard bases are:

Basis Name	Basis States	How to Express |0>	How to Express |1>
Z-basis (computational)	{|0>, |1>}	|0> itself	|1> itself
X-basis (Hadamard)	{|+>, |->}	|0> = (|+>+|->) / sqrt(2)	|1> = (|+>-|->) / sqrt(2)
Y-basis	{|i+>, |i->}	|0> = (|i+>+|i->) / sqrt(2)	|1> = -i(|i+>-|i->) / sqrt(2)

To measure in a different basis: apply the transformation that maps the desired basis to the Z-basis, then measure. For X-basis measurement: apply H, then measure in Z-basis.

5.5 CHSH Game — Quantum Advantage

The CHSH (Clauser-Horne-Shimony-Holt) game demonstrates quantum advantage over classical strategies. Two players (Alice and Bob) are separated and each receives a bit (x for Alice, y for Bob). They win if $a \oplus b = x \wedge y$ (their outputs XOR equals AND of inputs). They cannot communicate during the game.

- Classical deterministic strategy: maximum winning probability = 3/4 (75%)
- Classical probabilistic strategy: also maximum 3/4 — no classical strategy does better
- Quantum strategy using EPR pair: winning probability = cos^2(pi/8) ≈ 0.854 (85.4%)
- This proves quantum correlations are fundamentally stronger than classical correlations

TOPIC	CHAPTER 5 — QUANTUM COMMUNICATION

DEFINITION	Quantum teleportation sends quantum states using 2 classical bits + EPR pair. Superdense coding sends 2 classical bits using 1 qubit + EPR pair.
KEY FORMULA / RULE	Teleportation correction: {00→I, 01→X, 10→Z, 11→XZ}; Superdense coding: {00→I, 01→X, 10→Z, 11→XZ}
TYPES / VARIANTS	Quantum Teleportation, Superdense Coding, Bell States, EPR pairs, CHSH game, Quantum basis
KEY DIAGRAM	Teleportation circuit (CNOT + H + measurement + correction); Superdense coding circuit
MOST LIKELY EXAM Q	Explain quantum teleportation with full mathematical derivation. OR Compare quantum teleportation and superdense coding. OR Derive Bell state creation circuit.
REMEMBER	Teleportation sends QUANTUM STATE using CLASSICAL channel. Superdense sends CLASSICAL BITS using QUANTUM channel. They are duals of each other!

CHAPTER 6: QUANTUM FOURIER TRANSFORM (QFT)

The Quantum Fourier Transform (QFT) is the quantum analogue of the classical Discrete Fourier Transform (DFT). The QFT is used as a key subroutine in many important quantum algorithms including Shor's factoring algorithm and the quantum phase estimation algorithm. The QFT transforms from the computational basis to the Fourier basis.

6.1 QFT Definition and Formula

The QFT on an n-qubit system (N = 2^n states, j from 0 to N-1) is defined by:

QFT DEFINITION — MEMORIZE THIS

QFT|j> = (1/sqrt(N)) * sum_{k=0}^{N-1} e^{2*pi*i*j*k/N} |k>

= (1/2^{n/2}) * sum_{k=0}^{2^n-1} e^{2*pi*i*j*k/2^n} |k>

Equivalently with omega = e^{2*pi*i/N} = N-th root of unity:

QFT|j> = (1/sqrt(N)) * sum_{k=0}^{N-1} omega^{jk} |k>

QFT MATRIX: F_N(j,k) = omega^{jk} / sqrt(N)   (entry at row j, column k)

F_N is always a UNITARY matrix — so QFT is a valid quantum gate

The QFT is the QUANTUM version of the DFT (Discrete Fourier Transform)

Classical DFT: O(N log N) using FFT; Quantum QFT: O(n^2) = O(log^2 N) — exponentially faster

6.2 QFT as Product of Single-Qubit Operations

The QFT can be rewritten as a tensor product of individual qubit operations, which is what makes it efficiently implementable on a quantum computer. Using binary representation of j = j_1 j_2 ... j_n and k = k_1 k_2 ... k_n:

QFT PRODUCT REPRESENTATION

QFT|j> = (1/2^{n/2}) * prod_{l=1}^{n} (|0> + e^{2*pi*i*0.j_{n-l+1}...j_n} |1>)

This means QFT|j> = tensor product of n single-qubit states

Each qubit l gets: (|0> + e^{2*pi*i*(fractional part of j/2^l)} |1>) / sqrt(2)

The fractional binary representation: 0.j_l = j_l/2 + j_{l+1}/4 + ... + j_n/2^{n-l+1}

R_k gate (phase rotation gate): [[1,0],[0,e^{2*pi*i/2^k}]]

R_k|0> = |0>, R_k|1> = e^{2*pi*i/2^k}|1>

R_1 = Z gate (phase pi), R_2 = S gate (phase pi/2), R_3 = T gate (phase pi/4)

6.3 QFT Matrix for N=4

For N = 4 (n = 2 qubits), omega = e^{2*pi*i/4} = e^{pi*i/2} = i (because i = e^{pi*i/2}):

QFT MATRIX F_4 (N=4, omega=i)
omega = e^{2*pi*i/4} = i
omega^2 = e^{pi*i} = -1
omega^3 = e^{3*pi*i/2} = -i
omega^4 = e^{2*pi*i} = 1 (resets to 1)
F_4 = (1/2) * [[1, 1, 1, 1],
[1, i, -1, -i],
[1,-1, 1, -1],
[1,-i, -1, i]]
Entry F_4(j,k) = omega^{jk}/sqrt(4) = i^{jk}/2 F_4 is unitary: F_4 * F_4† = I ✓

6.4 QFT Matrix for N=8

For N = 8 (n = 3 qubits), omega = e^{2*pi*i/8} = e^{pi*i/4}. This requires knowing:

- omega = e^{pi*i/4} = cos(pi/4) + i*sin(pi/4) = (1+i)/sqrt(2)
- omega^2 = e^{pi*i/2} = i
- omega^3 = e^{3*pi*i/4} = (-1+i)/sqrt(2) = -i (wait, need exact: e^{3pi*i/4} = cos(3pi/4)+i*sin(3pi/4) = -1/sqrt2 + i/sqrt2)
- omega^4 = e^{pi*i} = -1
- omega^6 = (omega^3)^2; omega^7 = omega^4 * omega^3

SOLVED NUMERICAL — Computing QFT|j> for Specific State
PROBLEM: Compute QFT|3> for N=8 system. Write the result as superposition of basis states.
GIVEN: j=3, N=8 (n=3 qubits), omega = e^{2*pi*i/8} = e^{pi*i/4}
FIND: QFT|3> = sum of omega^{3k}|k> / sqrt(8) for k=0 to 7
FORMULA: QFT|j> = (1/sqrt(N)) sum_{k=0}^{N-1} omega^{jk} |k>
STEP 1: j=3, N=8: QFT|3> = (1/sqrt(8)) sum_{k=0}^{7} omega^{3k} |k>

STEP 2: k=0: omega^0 = 1 → coefficient of |0> is 1/sqrt(8)
STEP 3: k=1: omega^3 = e^{3*pi*i/4} = (-1+i)/sqrt(2) → coefficient of |1> is omega^3/sqrt(8)
STEP 4: k=2: omega^6 = e^{6*pi*i/4} = e^{3*pi*i/2} = -i → coefficient of |2>
STEP 5: k=3: omega^9 = omega^9 mod(period 8) = omega^1 = omega
STEP 6: k=4: omega^12 = omega^4 = e^{pi*i} = -1
STEP 7: k=5: omega^15 = omega^7 = e^{7*pi*i/4} = (1-i)/sqrt(2)
STEP 8: k=6: omega^18 = omega^2 = i
STEP 9: k=7: omega^21 = omega^5 = e^{5*pi*i/4} = (-1-i)/sqrt(2)
FINAL ANSWER: QFT|3> = (1/sqrt(8))(|0> + omega^3|1> + omega^6|2> + omega|3> + (-1)|4> + omega^7|5> + i|6> + omega^5|7>)
VERIFY: Use omega = e^{pi*i/4}, omega^2=i, omega^4=-1, omega^8=1 to simplify. QFT|3> is spread over all 8 basis states.

6.5 QFT Circuit

The QFT on n qubits requires O(n^2) gates: n Hadamard gates and O(n^2/2) controlled phase rotation gates R_k. The circuit applies H followed by conditional R_k rotations, building up the phase of each qubit.

QFT CIRCUIT FOR 3 QUBITS (|j1 j2 j3>)
Input: |j1>|j2>|j3> (3 qubit register)
Step 1: Apply H to |j1> → (|0>+e^{2*pi*i*0.j1}|1>)/sqrt(2)
Step 2: Apply R2 controlled on |j2> → adds phase e^{2*pi*i*j2/4} Step 3: Apply R3 controlled on |j3> → adds phase e^{2*pi*i*j3/8}
|j1> wire: now has (|0>+e^{2*pi*i*0.j1j2j3}|1>)/sqrt(2) Step 4: Apply H to |j2>
Step 5: Apply R2 controlled on |j3> to |j2> |j2> wire: now has (|0>+e^{2*pi*i*0.j2j3}|1>)/sqrt(2)
Step 6: Apply H to |j3>
|j3> wire: now has (|0>+e^{2*pi*i*0.j3}|1>)/sqrt(2)
FINAL STEP: Swap |j1> and |j3> to correct bit order (QFT gives reversed order)

Output: QFT|j1 j2 j3> in correct order

6.5.1 QFT|101> — Full Calculation (From Professor Notes, 6/4/26)

This is a frequently asked numerical. Compute QFT|101> for N=8 (j=5 in decimal, since 101 in binary = 4+0+1 = 5):

SOLVED NUMERICAL — QFT|101> for N=8 System

PROBLEM: Compute QFT|101> for N=8 (3-qubit system). j=101 in binary = 5 in decimal.

GIVEN: j = 5 (decimal), N=8, n=3 qubits, omega = e^{2*pi*i/8}

FIND: QFT|101> as superposition of 8 basis states

FORMULA: QFT|j> = F_8 * |j> where F_8 is the 8x8 QFT matrix

STEP 1: j=5 in binary = 101, so input vector has 1 in position 5 (0-indexed) and 0 elsewhere

STEP 2: QFT|101> = F_8 * [0,0,0,0,0,1,0,0]^T

STEP 3: = (1/sqrt(8)) * [row 0 to 7 of F_8, column 5]

STEP 4: = (1/sqrt(8)) * [omega^{0*5}, omega^{1*5}, omega^{2*5}, omega^{3*5}, omega^{4*5}, omega^{5*5}, omega^{6*5}, omega^{7*5}]

STEP 5: = (1/sqrt(8)) * [1, omega^5, omega^10, omega^15, omega^20, omega^25, omega^30, omega^35]

STEP 6: Simplify using omega^8 = 1: omega^10=omega^2=i, omega^15=omega^7, omega^20=omega^4=-1

STEP 7: omega^25=omega^1=omega, omega^30=omega^6, omega^35=omega^3

STEP 8: = (1/sqrt(8)) [1, omega^5, i, omega^7, -1, omega, omega^6, omega^3]

STEP 9: In ket notation: (1/sqrt8)(|0>+omega^5|1>+i|2>+omega^7|3>-|4>+omega|5>+omega^6|6>+omega^3|7>)

FINAL ANSWER: QFT|101> = (1/sqrt(8))[1,omega^5,i,omega^7,-1,omega,omega^6,omega^3]^T where omega=e^{pi*i/4}

VERIFY: Can also express omega^5=(−1−i)/sqrt(2), omega^7=(1−i)/sqrt(2), etc. using Euler's formula.

6.6 Inverse QFT

The Inverse Quantum Fourier Transform (QFT†) maps from the Fourier basis back to the computational basis. Since QFT is unitary, its inverse is simply its conjugate transpose: QFT† = QFT^{-1}.

- Circuit: Run QFT circuit in REVERSE, with each gate replaced by its conjugate
- H gate is self-inverse: H† = H
- R_k gate inverse: R_k† = [[1,0],[0,e^{-2*pi*i/2^k}]] (negate the phase)

- Used in Shor's algorithm and quantum phase estimation to extract phase information

TOPIC	CHAPTER 6 — QUANTUM FOURIER TRANSFORM
DEFINITION	QFT transforms computational basis to Fourier basis using complex phase rotations; efficiently implementable in O(n^2) gates.
KEY FORMULA / RULE	QFT|j> = (1/sqrt(N)) sum_k omega^{jk}|k>; omega = e^{2*pi*i/N}; F_N(j,k) = omega^{jk}/sqrt(N)
TYPES / VARIANTS	QFT definition, QFT matrix F_N, Omega (N-th root of unity), R_k gate, QFT circuit, QFT^{-1}
KEY DIAGRAM	QFT circuit for 3 qubits (H, R2, R3, H, R2, H, SWAP); F_4 and F_8 matrices
MOST LIKELY EXAM Q	Compute QFT|j> for a given j and N. OR Draw the QFT circuit for n=3 qubits and show state evolution. OR Compute F_4 matrix explicitly.
REMEMBER	omega = e^{2*pi*i/N}. For N=4: omega=i. For N=8: omega=e^{pi*i/4}=(1+i)/sqrt(2). ALWAYS simplify omega^k using omega^N=1.

CHAPTER 7: EIGENVALUES AND EIGENVECTORS IN QUANTUM COMPUTING

Eigenvalues and eigenvectors are fundamental concepts in linear algebra with direct quantum mechanical significance. In quantum mechanics, quantum states that are 'stable' under a particular operation (don't change direction, only scale) are eigenstates. The scaling factors are eigenvalues, which correspond to measurable physical quantities.

7.1 Definitions

EIGENVALUE / EIGENVECTOR DEFINITIONS
For a matrix A, a non-zero vector v is an EIGENVECTOR if: A*v = lambda*v
The scalar lambda is the EIGENVALUE corresponding to eigenvector v
Meaning: The matrix A stretches/scales v by factor lambda without changing its direction
In quantum mechanics: |psi> is an eigenstate of operator A if A|psi> = lambda|psi>
lambda = eigenvalue = the measurement outcome when measuring observable A in state |psi> CHARACTERISTIC EQUATION: det(A - lambda*I) = 0 gives all eigenvalues
For each eigenvalue, solve (A - lambda*I)v = 0 to find eigenvectors An n×n matrix has exactly n eigenvalues (counting multiplicity, in complex field)

7.2 Computing Eigenvalues — Step by Step

SOLVED NUMERICAL — Eigenvalues of Matrix A
PROBLEM: Find eigenvalues and eigenvectors of A = [[2,0],[0,1]].
GIVEN: A = [[2,0],[0,1]]
FIND: Eigenvalues lambda and eigenvectors v
FORMULA: Characteristic equation: det(A - lambda*I) = 0
STEP 1: A - lambda*I = [[2-lambda, 0],[0, 1-lambda]]
STEP 2: det(A - lambda*I) = (2-lambda)(1-lambda) - 0 = (2-lambda)(1-lambda)
STEP 3: Set equal to zero: (2-lambda)(1-lambda) = 0
STEP 4: Eigenvalues: lambda = 2 and lambda = 1
STEP 5: For lambda=1: solve (A-I)v = [[1,0],[0,0]]v = 0 → [x,0]^T=0 → x=0, y is free

STEP 6: Set y=1: eigenvector v1 = [0,1]^T
STEP 7: For lambda=2: solve (A-2I)v = [[0,0],[0,-1]]v = 0 → [0,-y]^T=0 → y=0, x is free
STEP 8: Set x=1: eigenvector v2 = [1,0]^T
STEP 9: Verification: A*v1 = [[2,0],[0,1]][0,1]^T = [0,1]^T = 1*v1 ✓
STEP 10: Verification: A*v2 = [[2,0],[0,1]][1,0]^T = [2,0]^T = 2*v2 ✓
FINAL ANSWER: lambda=1 with eigenvector [0,1]^T; lambda=2 with eigenvector [1,0]^T
VERIFY: Eigenvalues on diagonal for diagonal matrix — always.

7.3 Eigenvalues of Pauli Gates

The eigenvalues of Pauli gates are always ±1. Their eigenvectors are specific quantum states that are particularly important in quantum information theory.

7.3.1 Pauli-X Eigenvalues and Eigenvectors

SOLVED NUMERICAL — Pauli X Gate Eigenvalues
PROBLEM: Find eigenvalues and eigenvectors of X = [[0,1],[1,0]] (Pauli X gate).
GIVEN: X = [[0,1],[1,0]]
FIND: Eigenvalues and eigenvectors of X
FORMULA: Characteristic equation: det(X - lambda*I) = 0
STEP 1: X - lambda*I = [[-lambda, 1],[1, -lambda]]
STEP 2: det = (-lambda)(-lambda) - (1)(1) = lambda^2 - 1 = 0
STEP 3: lambda = ±1
STEP 4: For lambda = +1: (X-I)v = [[-1,1],[1,-1]]v = 0
STEP 5: Equation: -x+y = 0 → x = y. Set x=1, y=1: v = [1,1]^T
STEP 6: Normalize: v1 = [1,1]^T/sqrt(2) = |+> state
STEP 7: For lambda = -1: (X+I)v = [[1,1],[1,1]]v = 0
STEP 8: Equation: x+y = 0 → x = -y. Set x=1, y=-1: v = [1,-1]^T
STEP 9: Normalize: v2 = [1,-1]^T/sqrt(2) = |-> state
FINAL ANSWER: Eigenvalues: lambda = +1 (eigenvector |+>) and lambda = -1 (eigenvector |->)

VERIFY: These are exactly the Hadamard basis states! H maps Z-basis to X-basis and vice versa.

7.3.2 Pauli-Y Eigenvalues and Eigenvectors

SOLVED NUMERICAL — Pauli Y Gate Eigenvalues
PROBLEM: Find eigenvalues and eigenvectors of Y = [[0,-i],[i,0]] (Pauli Y gate).
GIVEN: Y = [[0,-i],[i,0]]
FIND: Eigenvalues and eigenvectors
FORMULA: Characteristic equation: det(Y - lambda*I) = 0
STEP 1: Y - lambda*I = [[-lambda,-i],[i,-lambda]]
STEP 2: det = (-lambda)(-lambda) - (-i)(i) = lambda^2 - i^2 = lambda^2 + 1... wait:
STEP 3: det = lambda^2 - (-i)(i) = lambda^2 - (-i^2) = lambda^2 - 1 = 0
STEP 4: Note: (-i)(i) = -i^2 = -(-1) = 1; so det = lambda^2 - 1 = 0
STEP 5: lambda = ±1
STEP 6: For lambda=+1: (Y-I)v = [[-1,-i],[i,-1]]v = 0
STEP 7: -x - iy = 0 → x = -iy. Set y=1: x = -i, v1 = [-i, 1]^T = [1/sqrt2][-i,1]^T → normalized
STEP 8: For lambda=-1: (Y+I)v = [[1,-i],[i,1]]v = 0
STEP 9: x - iy = 0 → x = iy. Set y=1: x = i, v2 = [i,1]^T → normalize: [i,1]^T/sqrt(2)
FINAL ANSWER: Eigenvalues: +1 (eigenvector [1,i]^T/sqrt(2)) and -1 (eigenvector [1,-i]^T/sqrt(2))
VERIFY: These are the Y-basis states |i+> and |i->.

7.3.3 Pauli-Z Eigenvalues and Eigenvectors

The Z gate [[1,0],[0,-1]] is diagonal, so its eigenvalues are directly on the diagonal: +1 and -1. The eigenvectors are the standard basis states |0> and |1>.

- lambda = +1: eigenvector = |0> = [1,0]^T (Z|0> = 1*|0> = |0>)
- lambda = -1: eigenvector = |1> = [0,1]^T (Z|1> = -1*|1> = -|1>)

This is why computational basis measurements are called 'Z-basis measurements' — because |0> and |1> are the eigenstates of the Z gate.

7.4 Summary of Pauli Gate Eigenstates

Pauli Gate	Matrix	Eigenvalue +1 Eigenstate	Eigenvalue -1 Eigenstate	Note

<table>
<tr><td>X (NOT)</td><td>[[0,1],[1,0]]</td><td>|+> = (|0>+|1>)/sqrt(2)</td><td>|-> = (|0>-|1>)/sqrt(2)</td><td>Hadamard basis states</td></tr>
<tr><td>Y</td><td>[[0,-i],[i,0]]</td><td>|i+> = (|0>+i|1>)/sqrt(2)</td><td>|i-> = (|0>-i|1>)/sqrt(2)</td><td>Y-basis states</td></tr>
<tr><td>Z</td><td>[[1,0],[0,-1]]</td><td>|0> = [1,0]^T</td><td>|1> = [0,1]^T</td><td>Computational basis states</td></tr>
</table>

7.5 Quantum Mechanical Significance of Eigenvalues

In quantum mechanics, when we measure a physical observable represented by a Hermitian (self-adjoint) matrix A on a quantum state |psi>:

- The possible measurement outcomes are the EIGENVALUES of A
- After measurement gives eigenvalue lambda_k, the state collapses to eigenvector |v_k>
- Probability of getting eigenvalue lambda_k = |<v_k|psi>|^2
- Expected value of measurement = <psi|A|psi> = sum_k lambda_k * P(lambda_k)
- Pauli gates X, Y, Z are all Hermitian: eigenvalues are ±1 (real, physical observables)
- Quantum gates must be UNITARY (not just Hermitian) to preserve probability

<table>
<tr><td>TOPIC</td><td>CHAPTER 7 — EIGENVALUES & EIGENVECTORS</td></tr>
<tr><td>DEFINITION</td><td>Eigenvalues are measurement outcomes; eigenvectors are stable states under quantum operators. For Av=lambda*v, det(A-lambda*I)=0 gives eigenvalues.</td></tr>
<tr><td>KEY FORMULA / RULE</td><td>det(A-lambda*I)=0; P(lambda_k) = |<v_k|psi>|^2; <psi|A|psi> = expected value</td></tr>
<tr><td>TYPES / VARIANTS</td><td>Characteristic equation, Eigenvalue, Eigenvector, Eigenstate, Pauli eigenstates (|+>,|->,|i+>,|i->)</td></tr>
<tr><td>KEY DIAGRAM</td><td>Vector diagram showing eigenvector direction unchanged by matrix; eigenvalue stretching</td></tr>
<tr><td>MOST LIKELY EXAM Q</td><td>Find eigenvalues and eigenvectors of all Pauli gates (X, Y, Z). OR Explain the quantum mechanical significance of eigenvalues and measurement outcomes.</td></tr>
<tr><td>REMEMBER</td><td>For Pauli gates X, Y, Z: eigenvalues are always ±1. Eigenvectors of Z are |0>,|1>; eigenvectors of X are |+>,|-> (Hadamard basis).</td></tr>
</table>

MASTER QUICK REVISION SHEET

Use this consolidated reference table in the last 30 minutes before the exam. Every topic, formula, and likely question summarized.

Topic	One-Line Definition	Key Formula/Rule	Most Likely Exam Question
Quantum Computing	Uses quantum mechanics (superposition, entanglement, interference) for computation	$P(\text{outcome}) = \lvert\text{amplitude}\rvert^2$	Compare classical vs quantum computing using double slit experiment
Qubit	2-state quantum system in superposition of $\lvert 0\rangle$ and $\lvert 1\rangle$	$\lvert\psi\rangle=c_0\lvert 0\rangle+c_1\lvert 1\rangle$; $\lvert c_0\rvert^2+\lvert c_1\rvert^2=1$	Normalize a given qubit state. Compute measurement probabilities.
Superposition	Quantum state simultaneously in multiple classical states	$H\lvert 0\rangle = (\lvert 0\rangle+\lvert 1\rangle)/\sqrt{2}$	Explain superposition with Schrodinger's cat analogy
Interference	Complex amplitudes add; result can be less than individual amplitudes	$\lvert c_1+c_2\rvert^2$ can be $< \lvert c_1\rvert^2$ or $\lvert c_2\rvert^2$	Explain destructive interference with numerical example
Deterministic System	One outgoing edge per state; matrix M has one 1 per column	$M*x$ = new state; M^2 = two steps	Write matrix for given system dynamics and compute state after 2 steps
Stochastic/Probabilistic	Transition probabilities; column sums = 1 (doubly stoch: row sums = 1 too)	$M*x$=new probabilities; column/row sums=1	Verify doubly stochastic property; compute probability distribution after n steps
Unitary Matrix	Quantum operator preserving norm; $U*U^\dagger = I = U^\dagger*U$	$U^\dagger$ = complex conjugate transpose	Verify a given matrix is unitary. Compute $U^\dagger$.
Inner Product	Measure of overlap between two quantum states; gives scalar	$\langle\phi\vert\psi\rangle = \text{sum}_k (\phi_k)^* * \psi_k$	Compute inner product of two states. Show basis states are orthogonal.

<table>
<tr><td>Outer Product</td><td>Matrix formed from ket times bra; used in projectors</td><td>|psi><phi| = column times row</td><td>Compute |0><0|, |1><1|, |0><1|. Show completeness relation.</td></tr>
<tr><td>Tensor Product</td><td>Combines multi-qubit states; dimension multiplies</td><td>[a,b]^T ⊗ [c,d]^T = [ac,ad,bc,bd]^T</td><td>Compute tensor product. Show that |Phi+> is entangled (cannot factor).</td></tr>
<tr><td>Entanglement</td><td>Multi-qubit state not separable as tensor product of individual qubits</td><td>Prove by contradiction: assume separable, derive contradiction</td><td>Prove |00>+|11> is entangled. Explain what entanglement means physically.</td></tr>
<tr><td>Measurement</td><td>Collapses superposition; Born rule gives probability of each outcome</td><td>P(xi)=|ci|^2/sum|ck|^2; state collapses to |xi></td><td>Compute probability of each measurement outcome for given state</td></tr>
<tr><td>X Gate (NOT)</td><td>Bit flip gate; maps |0>→|1> and |1>→|0></td><td>X=[[0,1],[1,0]]; X^2=I</td><td>Write matrix, show action on |0> and |1>, find eigenvalues</td></tr>
<tr><td>Y Gate</td><td>Bit flip + phase flip; flips with imaginary phase</td><td>Y=[[0,-i],[i,0]]; Y|0>=i|1></td><td>Show Y is unitary. Find eigenstates.</td></tr>
<tr><td>Z Gate</td><td>Phase flip only; |0>→|0>, |1>→-|1></td><td>Z=[[1,0],[0,-1]]; Z^2=I</td><td>Apply Z to superposition. Explain phase flip vs bit flip.</td></tr>
<tr><td>Hadamard Gate</td><td>Creates superposition from basis states; maps Z-basis to X-basis</td><td>H=(1/sqrt2)[[1,1],[1,-1]]; H^2=I</td><td>Apply H^n to |0...0>. Show H creates uniform superposition.</td></tr>
<tr><td>S and T Gates</td><td>Phase rotation gates; S adds pi/2, T adds pi/4 phase to |1></td><td>S=[[1,0],[0,i]]; T=[[1,0],[0,e^{i*pi/4}]]</td><td>Which gates form a universal quantum gate set? Explain T gate role.</td></tr>
<tr><td>CNOT Gate</td><td>Two-qubit gate; flips target iff control is |1></td><td>CNOT|10>=|11>, CNOT|11>=|10>; other pairs unchanged</td><td>Apply CNOT to create Bell state. Write CNOT matrix. Truth table.</td></tr>
<tr><td>Bell States</td><td>Four maximally entangled two-qubit</td><td>|Phi+>=(|00>+|11>)/sqrt(2); created by H then CNOT</td><td>Draw circuit to create |Phi+>. Show all four Bell</td></tr>
</table>

	states forming basis for 2-qubit space		states. Verify orthogonality.
Toffoli Gate	3-qubit CCNOT; flips target iff BOTH controls are |1>; reversible & universal	|x,y,z>→|x,y,z⊕(x∧y)>	Explain Toffoli universality. Show how AND and NOT are implemented.
No-Cloning Theorem	Impossible to copy arbitrary unknown quantum state	If C|x>|0>=|x>|x> then C cannot be linear (proof by contradiction)	Prove the no-cloning theorem. Explain implications for quantum cryptography.
Grover's Algorithm	Quantum search in O(sqrt(N)) — quadratic speedup over classical O(N)	Iterations = sqrt(N); Diffusion = -I+2A	Describe Grover's algorithm with all steps. Show inversion about mean numerically.
Simon's Algorithm	Find period of 2-to-1 function exponentially faster than classical	f(x)=f(y) iff x=y⊕c; quantum gives z·c=0 equations	Explain Simon's algorithm and its quantum advantage over classical approaches.
Quantum Teleportation	Send quantum state using 2 classical bits + EPR pair; original destroyed	Correction: {00→I, 01→X, 10→Z, 11→XZ}	Derive teleportation protocol with all mathematical steps. Draw circuit.
Superdense Coding	Send 2 classical bits using 1 qubit + EPR pair	Encode: {00→I, 01→X, 10→Z, 11→XZ} applied to Alice's EPR qubit	Describe superdense coding. Compare with teleportation (they are dual protocols).
QFT	Transform from computational to Fourier basis; key subroutine in Shor's algorithm	QFT|j>=(1/sqrt(N))sum_k omega^{jk}|k>; omega=e^{2*pi*i/N}	Compute QFT|j> for given j,N. Draw QFT circuit for n=3. Write F_4 matrix.
QFT Matrix F_N	N×N unitary matrix; F_N(j,k) = omega^{jk}/sqrt(N)	For N=4: omega=i; for N=8: omega=e^{pi*i/4}	Compute F_4 explicitly. Verify it is unitary. Compute QFT|101> for N=8.
Eigenvalues	Scaling factors for eigenvectors;	det(A-lambda*I)=0; Av=lambda*v	Find eigenvalues and eigenvectors

	measurement outcomes in quantum mechanics		of all Pauli gates X, Y, Z.
Pauli Eigenstates	Eigenstates of X are |+>,|->; of Y are |i+>,|i->; of Z are |0>,|1>	X: lambda=±1; Y: lambda=±1; Z: lambda=±1	Which states are eigenstates of Z? Of X? What are their eigenvalues?

IMPORTANT FORMULAS REFERENCE CARD

Formula Name	Formula	When to Use
Qubit state	|psi> = c0|0> + c1|1> = [c0, c1]^T	Define any single qubit
Normalization	|c0|^2 + |c1|^2 = 1	Check/verify quantum state validity
Measurement probability	P(xi) = |ci|^2 / sum_k|ck|^2	Find probability of each outcome
Inner product	<phi|psi> = sum_k (phi_k)* * psi_k	Overlap, transition amplitude
Norm (length)	||psi>|| = sqrt(<psi|psi>)	Compute vector length
Unitary condition	U*U† = U†*U = I	Check if gate is valid quantum gate
Conjugate transpose	U†: transpose then conjugate each entry	Compute U-dagger
Born rule	P(outcome) = |amplitude|^2	Probability from amplitude
Tensor product 2D	[a,b]^T ⊗ [c,d]^T = [ac,ad,bc,bd]^T	Combine two qubit states
Hadamard	H = (1/sqrt2)[[1,1],[1,-1]]	Create superposition
X gate	X = [[0,1],[1,0]]	Bit flip (quantum NOT)
Y gate	Y = [[0,-i],[i,0]]	Bit + phase flip
Z gate	Z = [[1,0],[0,-1]]	Phase flip only
CNOT matrix	4x4: I2 on top, [[0,1],[1,0]] on bottom-right 2x2	Controlled NOT, entanglement
Bell state |Phi+>	(|00>+|11>)/sqrt(2)	EPR pair, teleportation, superdense
Grover diffusion	v'_i = 2*mean(v) - v_i = (-I+2A)v	Amplitude amplification
Grover iterations	sqrt(N) = sqrt(2^n)	How many Grover steps
Simon's XOR relation	f(x)=f(y) iff x = y XOR c	Identify period c

QFT definition	QFT|j> = (1/sqrt(N)) sum_k omega^{jk}|k>	Quantum Fourier Transform
QFT omega	omega = e^{2*pi*i/N} (N-th root of unity)	Phase factor in QFT
F_4 matrix	F_4 = (1/2)[[1,1,1,1],[1,i,-1,-i],[1,-1,1,-1],[1,-i,-1,i]]	N=4 QFT matrix
Eigenvalue equation	Av = lambda*v; det(A-lambda*I) = 0	Find eigenvalues
Teleportation correction	Alice 00→Bob I; 01→X; 10→Z; 11→XZ	Teleportation recovery
Superdense encoding	Alice 00→I; 01→X; 10→Z; 11→XZ on her EPR qubit	Superdense coding
Phase kickback	Uf|x>(|0>-|1>)/sqrt2 = (-1)^f(x)|x>(|0>-|1>)/sqrt2	Grover & other algorithms

EXAM TIPS AND COMMON MISTAKES

Things Examiners Always Check

- Always NORMALIZE your state before computing probabilities
- When computing U†: FIRST transpose the matrix, THEN conjugate each entry (order matters!)
- Tensor product: position of qubits matters — |01> ≠ |10>
- CNOT maps |10>→|11> and |11>→|10>; leaves |00> and |01> unchanged
- Bell state creation: H on first qubit then CNOT (not the other way around)
- In teleportation: Alice sends 2 classical bits (measurement results) to Bob
- QFT: omega = e^{2*pi*i/N}; for N=4, omega=i; for N=8, omega=e^{pi*i/4}
- Grover's needs sqrt(N) iterations — not sqrt(n) where n=log(N)
- Eigenvalue calculation: set det(A-lambda*I) = 0, not det(A) = 0
- Pauli gates all have eigenvalues ±1; eigenvectors of Z are |0>,|1>

Common Mistakes to AVOID

- MISTAKE: Thinking quantum superposition is the same as classical probability — it's NOT. Amplitudes are complex; probabilities are real and non-negative.
- MISTAKE: Forgetting to renormalize after partial measurement on multi-qubit system
- MISTAKE: Writing omega^k without simplifying using omega^N = 1 periodicity in QFT
- MISTAKE: Confusing teleportation (quantum state transfer) with superdense coding (classical bits via quantum channel)
- MISTAKE: Saying cloning is possible in quantum computing — the No-Cloning Theorem forbids it
- MISTAKE: Applying Grover's to structured search — it's only for UNSTRUCTURED database search
- MISTAKE: Forgetting that all quantum gates must be unitary (and therefore reversible)

- MISTAKE: Confusing inner product <phi|psi> (scalar) with outer product |psi><phi| (matrix)

Memory Mnemonics

MNEMONICS FOR KEY FORMULAS
'H makes you Happy (superposition)': H|0> = (|0>+|1>)/sqrt(2) — always creates equal superposition 'X eXchanges': X|0>=|1>, X|1>=|0> — the bit flip
'Z Zaps the |1>': Z|0>=|0>, Z|1>=-|1> — only |1> gets the negative sign 'CNOT flips only when Control is 1': control=0 means target stays; control=1 means target flips
'Teleportation: Quantum state IN, Classical bits OUT (2 of them)' 'Superdense: Classical bits IN (2), Quantum qubit OUT (1 qubit sent)'
'BELL = Hadamard then CNOT' — always in this order to create Bell states 'QFT omega = e^{2pi*i/N}': When N=4, omega=i (because e^{pi*i/2}=i)
'Grover = sqrt(N) steps': For N=2^n items, sqrt(2^n) = 2^{n/2} iterations
'Eigenvalue = det(A-lambdaI)=0': Characteristic equation — always starts here